InnovShop

InnovShop

Plateforme E-Commerce de Produits Technologiques

DOSSIER DE PROJET

Présenté par	Berben Mathias
Formation	Développeur Web et Web Mobile (DWWM)
Établissement	Global digital university
Site en ligne	berben.alwaysdata.net/innovshop
Dépôt GitHub	github.com/Mberben/innovshop
Tableau Kanban	trello.com/b/tRx29sT4/innovshop

Sommaire

1 Introduction

2 Présentation du projet

3 Gestion de projet

4 Technologies utilisées

5 Architecture et base de données

6 Fonctionnalités principales

7 Sécurité et bonnes pratiques

8 Démonstration visuelle

9 Tests

10 Déploiement

11 Bilan

12 Conclusion

1. Introduction

1.1 Contexte

InnovShop est une start-up spécialisée dans la vente de produits technologiques innovants : audio, montres connectées, drones, lunettes connectées, maison connectée. Face à une demande croissante, sa directrice, Madame Leroy, souhaite digitaliser ses ventes. Elle m'a confié, en tant que développeur, la réalisation de la première version d'une plateforme e-commerce.

Ce projet a été réalisé dans le cadre de ma formation Développeur Web et Web Mobile, avec le framework Symfony.

1.2 Problématique

Comment concevoir une boutique en ligne sécurisée et simple à utiliser, qui permette à la fois aux clients de commander facilement et à l'équipe d'InnovShop de gérer son catalogue et ses commandes sans compétence technique ?

1.3 Objectifs

- Proposer un catalogue clair, avec recherche, filtres et fiches produits détaillées (options de couleur ou de taille, images).
- Offrir un panier accessible en permanence et mis à jour en temps réel.
- Mettre en place un processus de commande en plusieurs étapes, réservé aux clients connectés, avec un email récapitulatif.
- Donner à chaque client un espace personnel : historique et suivi des commandes, profil, objectif de dépenses.
- Fournir aux administrateurs un back-office complet : produits, commandes, clients.
- Respecter les bonnes pratiques de Symfony et les règles de sécurité du web.
- Déployer l'application sur un serveur distant.

2. Présentation du projet

2.1 Les utilisateurs

Profil	Besoins principaux
Visiteur	Découvrir les produits, les comparer, remplir un panier, créer un compte
Client connecté	Commander, recevoir une confirmation, suivre ses commandes, gérer son profil et son adresse
Administrateur (équipe InnovShop)	Gérer le catalogue, traiter les commandes, consulter les clients

2.2 Périmètre fonctionnel

Front-office

- Page d'accueil essentiellement graphique : 3 derniers produits ajoutés et 3 produits à la une.
- Catalogue avec recherche par mot-clé, filtre par catégorie, fourchette de prix et tri.
- Fiche produit : description, caractéristiques, prix, galerie d'images, options (couleur, taille…). La photo s'adapte à l'option choisie.
- Panier accessible depuis toutes les pages, mis à jour sans rechargement (AJAX).
- Inscription et connexion par email et mot de passe sécurisé.
- Commande en trois étapes : vérification du panier, adresse de livraison, confirmation. Un email récapitulatif est envoyé.
- Espace client : tableau de bord, objectif de dépenses mensuel, historique et suivi des commandes, facture PDF, profil et mot de passe.

Back-office

- Tableau de bord : chiffre d'affaires sur 14 jours, commandes en attente, dernières commandes.
- Gestion des produits, de leurs options et de leurs images, et des catégories.
- Gestion des commandes : consultation, filtres, changement de statut (le client est prévenu par email).
- Gestion des clients : coordonnées, rôle administrateur, historique des commandes.

2.3 Hors périmètre de cette première version

Conformément au cahier des charges :

- pas de paiement en ligne : la commande est enregistrée avec le statut « En attente » ;
- pas de gestion des quantités : chaque ajout au panier crée une nouvelle ligne ;
- pas de gestion des stocks.

3. Gestion de projet

3.1 Analyse des besoins et backlog

J'ai d'abord analysé le cahier des charges pour le traduire en **30 user stories**, au format « En tant que… je veux… afin de… », chacune avec ses critères d'acceptation. Je les ai priorisées selon la méthode **MoSCoW** et estimées en **story points** (suite de Fibonacci).

Priorité	Nombre de user stories	Story points
Must (exigé par le cahier des charges)	23	68
Should (important)	3	6
Could (bonus)	4	9
Total	**30**	**83**

Exemple : *« En tant que visiteur, je veux ajouter un produit au panier sans recharger la page, afin de constituer ma commande de façon fluide. »* Critères d'acceptation : requête AJAX, une nouvelle ligne par ajout, compteur du panier mis à jour immédiatement.

Backlog du produit — InnovShop (plateforme e-commerce Symfony)

User stories	Rôles/Personas	Fonctions/Fonctionnalités	Raisons/Objectifs	Acceptation	Évaluation	Story point
1	En tant que visiteur	Je veux voir sur la page d'accueil les 3 derniers produits ajoutés et 3 produits à la une	Afin de découvrir rapidement les nouveautés et les produits mis en avant	L'accueil affiche les 3 produits les plus récents (tri par date de création) et 3 produits marqués « à la une » ; chaque carte mène à la fiche produit	Must	3
2	En tant que visiteur	Je veux consulter le catalogue de tous les produits	Afin de parcourir l'offre d'InnovShop	Grille responsive (image, nom, prix, bouton « Voir ») ; affichage correct sur mobile, tablette et desktop	Must	3
3	En tant que visiteur	Je veux consulter la fiche détaillée d'un produit	Afin de connaître sa description, ses spécifications, son prix et voir ses images	La page /produit/{id} affiche description, spécifications, prix, images ; 404 personnalisée si le produit n'existe pas	Must	3
4	En tant que visiteur	Je veux choisir une option (couleur, taille…) sur la fiche produit	Afin de acheter la variante qui me convient	Les options disponibles du produit sont proposées dans une liste ; l'option choisie est conservée dans le panier	Must	5
5	En tant que visiteur	Je veux rechercher un produit par mot-clé ou filtrer par catégorie	Afin de trouver plus vite ce que je cherche	La recherche porte sur le nom et la description ; le filtre par catégorie réduit la liste ; aucun résultat = message explicite	Could	3
6	En tant que visiteur	Je veux ajouter un produit au panier sans recharger la page	Afin de constituer ma commande de façon fluide	Requête AJAX ; chaque ajout crée une nouvelle ligne (pas de quantité) ; le compteur du panier se met à jour immédiatement	Must	5
7	En tant que visiteur	Je veux supprimer une ligne de mon panier	Afin de corriger ma sélection	La ligne disparaît sans rechargement ; le total est recalculé ; panier vide = message	Must	2
8	En tant que visiteur	Je veux accéder à mon panier depuis n'importe quelle page	Afin de suivre mon panier en permanence	Icône + compteur dans le header sur toutes les pages ; lien vers la page panier	Must	2
9	En tant que visiteur	Je veux créer un compte avec mon email et un mot de passe sécurisé	Afin de pouvoir passer commande	Email unique et valide ; mot de passe d'au moins 8 caractères, hashé ; formulaire protégé par CSRF ; messages d'erreur clairs	Must	3
10	En tant que visiteur	Je veux me connecter et me déconnecter	Afin de accéder à mon espace personnel	Connexion par email/mot de passe ; message en cas d'identifiants incorrects ; lien de déconnexion dans le header	Must	2
11	En tant que visiteur	Je veux être redirigé vers la connexion si je tente de commander sans être connecté	Afin de comprendre qu'un compte est nécessaire	L'accès au tunnel de commande exige ROLE_USER ; après connexion, retour vers le tunnel ; le panier est conservé	Must	1
12	En tant que client	Je veux vérifier mon panier (étape 1 de la commande)	Afin de m'assurer du contenu avant de commander	Récapitulatif des lignes (produit, option, prix) et du total ; bouton « Continuer » ; impossible avec un panier vide	Must	2
13	En tant que client	Je veux saisir ou confirmer mon adresse de livraison (étape 2)	Afin de recevoir ma commande au bon endroit	Adresse pré-remplie depuis le profil ; champs obligatoires validés (Assert) : nom, adresse, code postal à 5 chiffres, ville, téléphone	Must	3
14	En tant que client	Je veux confirmer ma commande (étape 3)	Afin de finaliser mon achat	Commande enregistrée avec ses lignes, l'adresse, le total et le statut « en attente » ; panier vidé ; page de confirmation avec la référence	Must	5
15	En tant que client	Je veux recevoir un email récapitulatif après ma commande	Afin de garder une trace de mon achat	Email envoyé via Symfony Mailer avec template Twig : référence, lignes, total, adresse	Must	3
16	En tant que client	Je veux consulter l'historique de mes commandes	Afin de suivre mes achats	Liste triée par date (référence, date, total, statut) ; je ne vois que mes propres commandes (Voter)	Must	2
17	En tant que client	Je veux voir le détail et le statut d'une commande	Afin de savoir où en est ma livraison	Lignes, prix, adresse, badge de couleur selon le statut ; accès refusé (403) à la commande d'un autre client	Must	2
18	En tant que client	Je veux modifier mes informations de profil et de livraison	Afin de garder des coordonnées à jour	Modification du nom, de l'adresse et du téléphone avec validation ; message de succès	Must	3
19	En tant que client	Je veux changer mon mot de passe	Afin de sécuriser mon compte	Ancien mot de passe vérifié ; nouveau mot de passe confirmé et hashé	Should	2
20	En tant que client	Je veux définir un objectif de dépenses mensuel et voir où j'en suis	Afin de maîtriser mon budget	Montant saisi dans le profil ; le tableau de bord affiche le total du mois et une barre de progression	Should	3
21	En tant que client	Je veux télécharger la facture d'une commande en PDF	Afin de conserver un justificatif	Bouton sur le détail de la commande ; PDF généré avec les lignes et le total	Could	3
22	En tant que client	Je veux être notifié par email quand le statut de ma commande change	Afin de suivre l'expédition sans me connecter	Email envoyé quand l'admin modifie le statut	Could	2
23	En tant que administrateur	Je veux accéder à un back office réservé aux administrateurs	Afin de gérer la boutique en toute sécurité	/admin protégé par ROLE_ADMIN (security.yaml + IsGranted) ; un client reçoit une erreur 403	Must	2
24	En tant que administrateur	Je veux ajouter, modifier et supprimer des produits	Afin de tenir le catalogue à jour	CRUD EasyAdmin : nom, description, spécifications, prix, images, catégorie, « à la une » ; champs validés	Must	5
25	En tant que administrateur	Je veux gérer les options d'un produit (couleur, taille…)	Afin de proposer des variantes	Ajout/suppression d'options rattachées à un produit depuis EasyAdmin	Must	3
26	En tant que administrateur	Je veux visualiser toutes les commandes et modifier leur statut	Afin de traiter et expédier les commandes	Liste des commandes avec filtre par statut ; statut modifiable (en attente, validée, expédiée, annulée)	Must	3
27	En tant que administrateur	Je veux visualiser les clients et leur historique de commandes	Afin de assurer le suivi client	Liste des utilisateurs ; la fiche d'un client affiche ses commandes	Must	3
28	En tant que administrateur	Je veux modifier les informations et le rôle d'un utilisateur	Afin de nommer un autre administrateur	Rôle modifiable depuis EasyAdmin	Should	1
29	En tant que Mme Leroy (cliente)	Je veux que la plateforme soit en ligne et documentée	Afin de la présenter et la faire évoluer	Application déployée (APP_ENV=prod) avec une URL publique ; README avec les étapes d'installation	Must	3
30	En tant que visiteur	Je veux voir des pages d'erreur 404 et 403 claires	Afin de ne pas être perdu en cas d'erreur	Templates d'erreur Twig personnalisés avec un lien de retour à l'accueil	Could	1

Total story points		83
dont Must	Must	68
dont Should	Should	6
dont Could	Could	9

Légende
Évaluation : priorité MoSCoW — Must (indispensable, exigé par le sujet), Should (important), Could (bonus si le temps le permet), Won't (hors V1).
Story points : estimation relative de l'effort sur l'échelle de Fibonacci (1, 2, 3, 5, 8).
Hors V1 (d'après le sujet) : paiement en ligne, gestion des quantités dans le panier, gestion des stocks.

Figure — Backlog du produit : 30 user stories avec leurs critères d'acceptation, leur priorité MoSCoW et leurs story points (83 au total, dont 68 en Must).

3.2 Organisation avec un tableau Kanban

J'ai suivi l'avancement sur un tableau **Trello** public, en quatre colonnes : Backlog, À faire, En cours, Fait. Chaque carte correspond à une user story, avec une étiquette de couleur selon sa priorité (vert : Must, jaune : Should, bleu : Could). J'ai ajouté des cartes techniques pour la mise en place de l'environnement et de la base de données. Je déplaçais chaque carte au moment où je commençais puis terminais la tâche ; l'historique d'activité de Trello en garde la trace.



Figure — Début du projet (27 octobre au matin) : les user stories sont ajoutées une par une au Backlog, puis le tableau est rendu public.

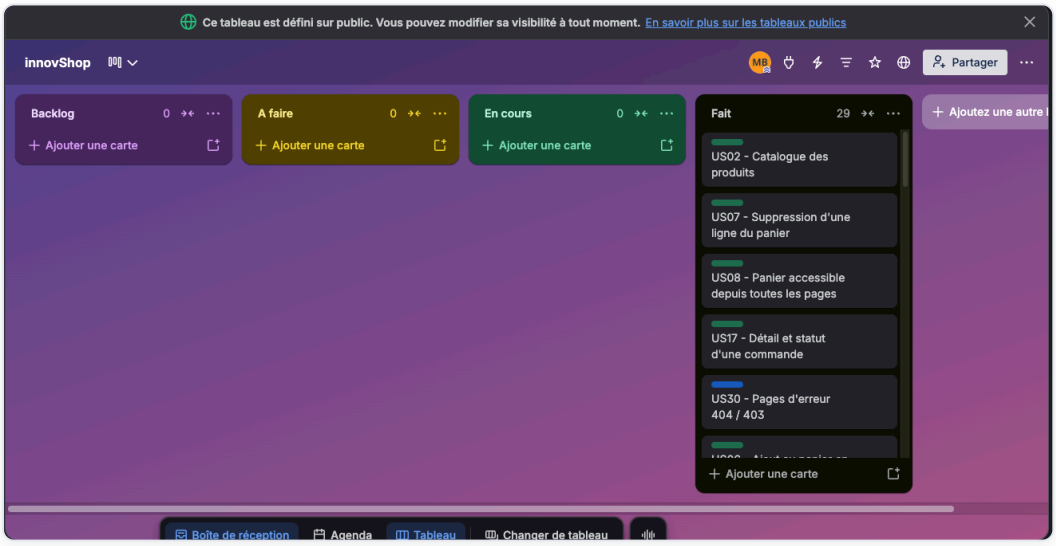


Figure — Tableau Trello public en fin de projet : les 29 cartes sont dans la colonne « Fait ».



Figure — Historique d'activité : chaque carte passe de « A faire » à « En cours » puis à « Fait », avec l'heure de chaque déplacement.

3.3 Planning

Phase	Contenu
Conception	Analyse du sujet, backlog, diagramme de classes, tableau Kanban
Mise en place	Environnement, projet Symfony, dépôt GitHub, base de données, entités et migrations
Développement back	Authentification, back-office EasyAdmin, données de démonstration
Développement front	Accueil, catalogue, fiche produit, panier AJAX, tunnel de commande, espace client, emails
Finitions	Photos produits, ajustements d'ergonomie, README, déploiement

3.4 Gestion des versions avec Git

Le code est versionné avec **Git** et hébergé sur **GitHub**. J'ai fait un commit à la fin de chaque étape fonctionnelle et testée, avec un message descriptif (par exemple « Entités Order, OrderLine et enum OrderStatus »). Les fichiers sensibles ou générés (`.env.local`, `vendor/`, `var/`, images uploadées) sont exclus par le `.gitignore`.

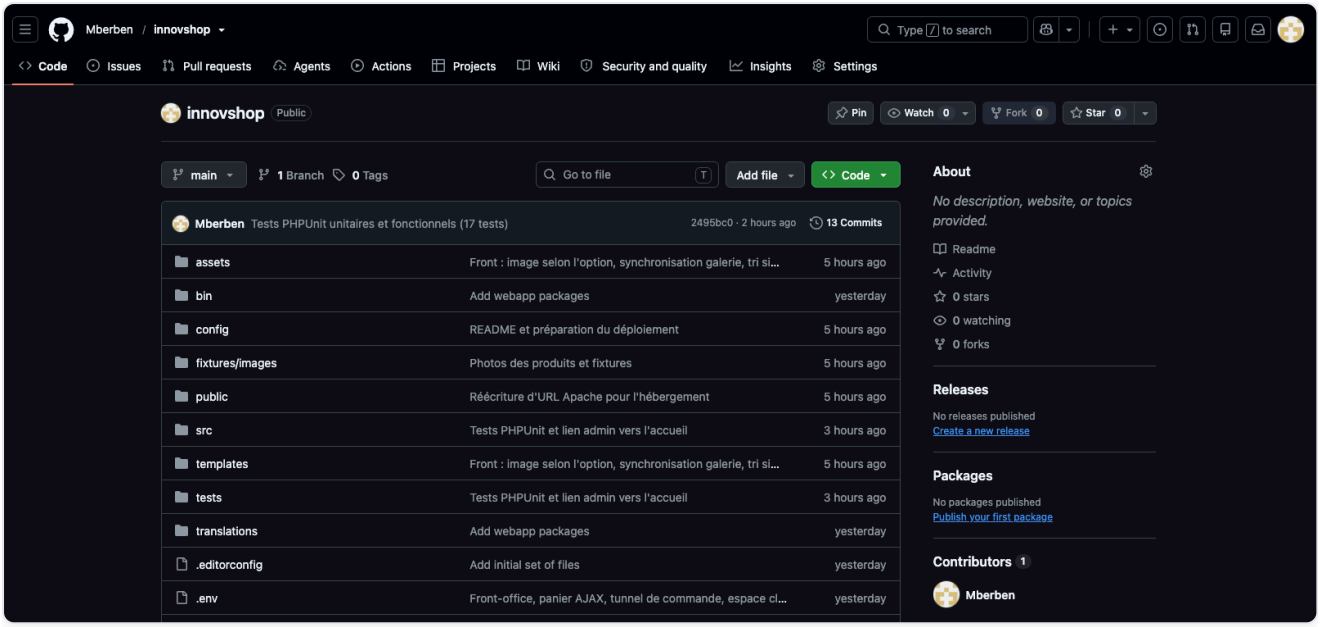


Figure — Dépôt public Mberben/innovshop sur GitHub : 13 commits sur la branche main, avec le dernier message de commit en face de chaque dossier.

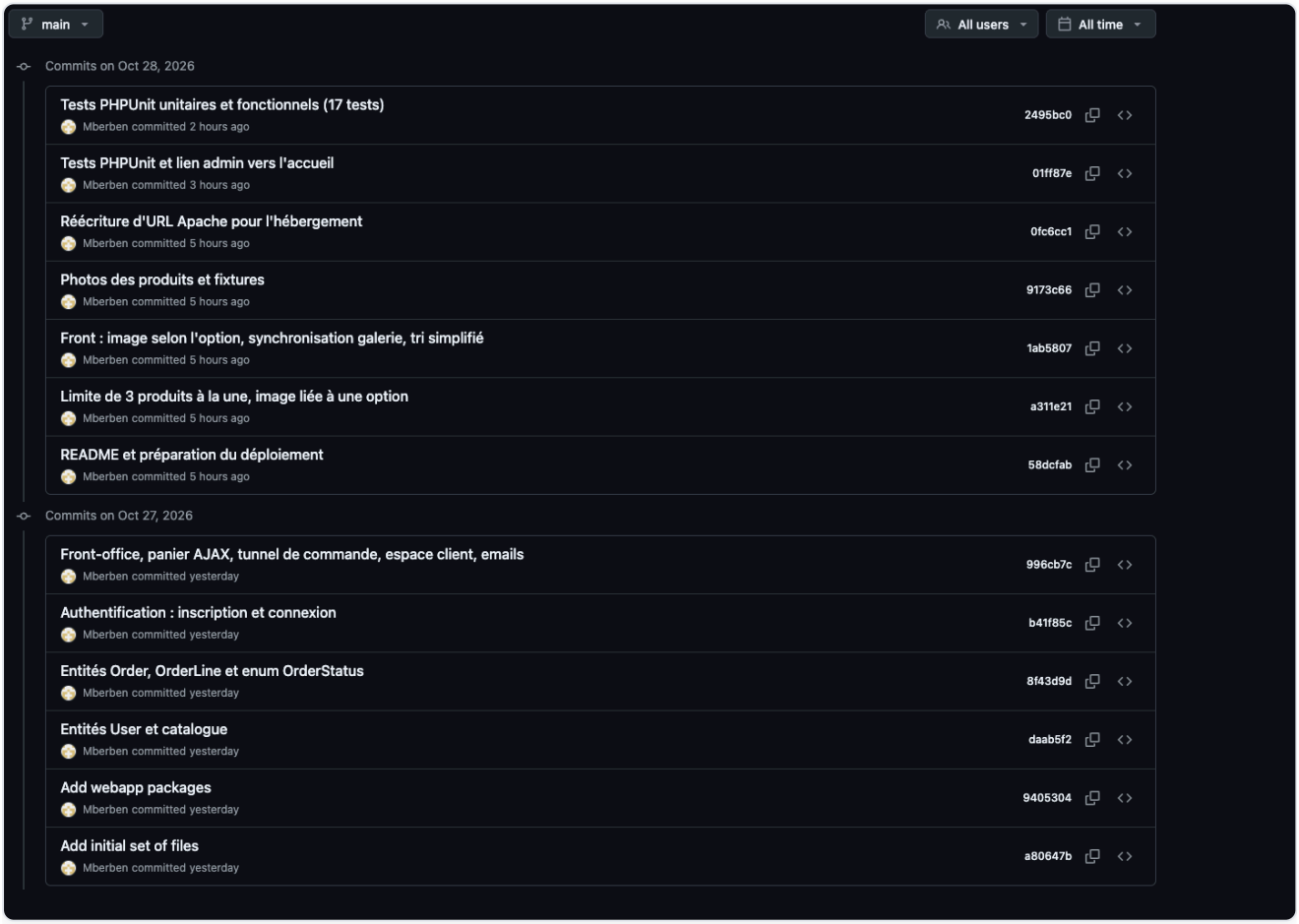


Figure — Historique des commits sur GitHub : un commit par étape (entités, authentification, front-office, déploiement, tests), répartis sur les 27 et 28 octobre.

4. Technologies utilisées

4.1 Stack technique

Domaine	Technologie	Rôle dans le projet
Langage serveur	PHP 8.5	Langage de l'application
Framework	Symfony 8.1	Structure MVC, routage, sécurité, formulaires, emails
ORM	Doctrine ORM 3	Correspondance entre les classes PHP et les tables, migrations
Base de données	MariaDB 10.4 (XAMPP en local)	Stockage des données
Templates	Twig	Génération des pages HTML
Interface	Bootstrap 5.3, Bootstrap Icons, CSS personnalisé	Mise en page responsive et design
JavaScript	JavaScript natif (fetch), AssetMapper	Panier AJAX, galerie, synchronisation des options
Back-office	EasyAdmin 5	Interface d'administration
Emails	Symfony Mailer, TemplatedEmail	Emails de confirmation et de suivi
PDF	Dompdf	Factures téléchargeables
Données de démo	DoctrineFixturesBundle	Jeu de données de test réaliste
Outils	VS Code, Symfony CLI, Composer, Git, GitHub, Trello	Développement, dépendances, versions, organisation
Hébergement	Alwaysdata	Mise en production

J'ai aussi utilisé un assistant d'intelligence artificielle (Claude) pour m'accompagner : explication des concepts, aide à la rédaction de code et au débogage. J'ai relu, testé et documenté chaque partie du code pour être capable de l'expliquer et de la faire évoluer.

4.2 Pourquoi Symfony ?

- **Structure claire** : le modèle MVC sépare les données, la logique et l'affichage, ce qui rend le code plus lisible et plus facile à maintenir.
- **Sécurité intégrée** : hachage des mots de passe, protection CSRF, gestion des rôles, voters.
- **Productivité** : le MakerBundle génère les entités, l'authentification et les formulaires ; EasyAdmin produit un back-office complet.

- **Écosystème reconnu** : documentation complète, nombreux composants, très utilisé en entreprise en France.

4.3 Choix d'AssetMapper plutôt que Webpack Encore

Les projets Symfony récents utilisent **AssetMapper** par défaut : les fichiers CSS et JavaScript sont servis directement par le navigateur, sans étape de compilation Node.js. Pour un site de cette taille, c'est plus simple à installer et à déployer, et c'est le choix recommandé par Symfony. En production, la commande `asset-map:compile` ajoute une empreinte au nom des fichiers pour permettre leur mise en cache par le navigateur.

5. Architecture et base de données

5.1 Architecture MVC

Symfony organise l'application selon le modèle **Modèle – Vue – Contrôleur**. Voici le parcours d'une requête, par exemple l'affichage de la fiche d'un produit :

1. Le navigateur demande `/produit/montre-vita-sport`.
2. Le **routeur** trouve la route correspondante (`#[Route('/produit/{slug}')]`) et appelle `ProductController::show()`.
3. Doctrine charge le **modèle** : l'entité `Product` dont le slug correspond, avec ses images et ses options. Si elle n'existe pas, Symfony renvoie une erreur 404.
4. Le **contrôleur** transmet le produit à la **vue** Twig `product/show.html.twig`.
5. Twig génère le HTML, qui est renvoyé au navigateur.

J'ai ajouté une couche de **services** entre les contrôleurs et les données pour la logique métier (panier, emails) : les contrôleurs restent courts et ne font que coordonner.

5.2 Organisation du code

Dossier	Contenu
`src/Entity/`	Les 7 entités Doctrine et leurs règles de validation
`src/Enum/`	`OrderStatus` : statuts de commande, libellés et couleurs
`src/Repository/`	Requêtes sur mesure (recherche du catalogue, statistiques)
`src/Controller/`	Contrôleurs du site : accueil, catalogue, panier, commande, espace client, authentification
`src/Controller/Admin/`	Tableau de bord et écrans de gestion EasyAdmin
`src/Service/`	`CartService` : logique du panier
`src/Mailer/`	`OrderMailer` : emails liés aux commandes
`src/EventListener/`	Email automatique au changement de statut
`src/Security/Voter/`	`OrderVoter` : accès aux commandes
`src/Validator/`	Contrainte `MaxFeatured` : 3 produits à la une maximum
`src/Form/`	Formulaires (inscription, livraison, profil, mot de passe)
`src/Twig/`	Filtre `price` et fonction `cart_count()`
`src/DataFixtures/`	Données de démonstration
`templates/`	Vues Twig, dont `_partials/` pour les éléments réutilisés
`assets/`	CSS et JavaScript

5.3 Diagramme de classes

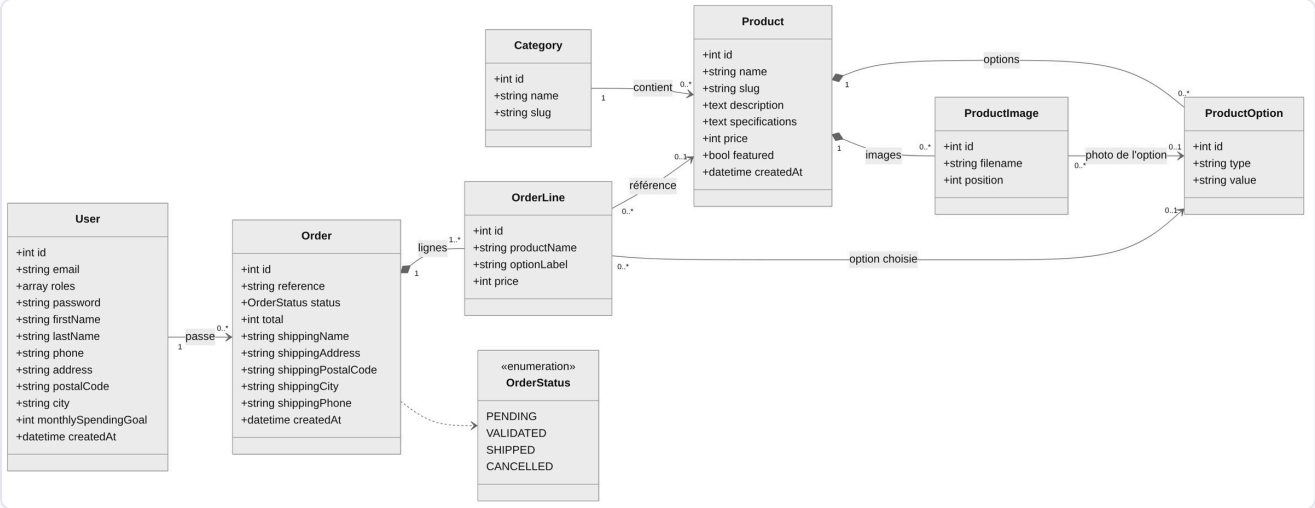


Figure — Diagramme de classes : 7 entités et l'énumération OrderStatus (prix et montants stockés en centimes, type int).

Entité	Rôle	Relations
`User`	Compte client ou administrateur : email, mot de passe haché, rôles, coordonnées, objectif de dépenses	1 utilisateur → 0..n commandes
`Category`	Catégorie de produits (nom, slug)	1 catégorie → 0..n produits
`Product`	Produit : nom, slug, description, caractéristiques, prix, « à la une »	1 produit → 0..n images, 0..n options
`ProductImage`	Photo d'un produit, avec son ordre d'affichage	Peut illustrer une option précise
`ProductOption`	Option d'un produit (type + valeur, ex. Couleur : Blanc)	Appartient à un produit
`Order`	Commande : référence, statut, total, adresse de livraison, date	1 commande → 1..n lignes
`OrderLine`	Ligne de commande : nom, option et prix copiés au moment de l'achat	Lien facultatif vers le produit et l'option

5.4 Choix de conception

- **Prix en centimes (entiers)** : 19,99 € est stocké `1999`. Les nombres à virgule flottante produisent des erreurs d'arrondi (en PHP, `0.1 + 0.2` donne `0.30000000000000004`) ; les entiers garantissent des totaux exacts. L'affichage passe par un filtre Twig `price`.
- **Données figées dans la commande** : chaque ligne copie le nom, l'option et le prix du produit, et la commande copie l'adresse de livraison. Modifier ou supprimer un produit, ou changer d'adresse, ne fausse donc jamais l'historique. Les liens vers le produit passent à NULL en cas de suppression (`onDelete: 'SET NULL'`).
- **Statut en énumération PHP** : `OrderStatus` n'accepte que 4 valeurs (en attente, validée, expédiée, annulée). Aucune valeur invalide ne peut être enregistrée.

- **Panier en session**, sans entité : il n'est enregistré en base qu'au moment de la commande.
- **Pas de champ quantité**, comme le demande le cahier des charges : une ligne par ajout.
- **Table `order`** : ORDER étant un mot réservé en SQL, Doctrine entoure le nom de la table de guillemets, et la relation inverse s'appelle `customerOrder` dans `OrderLine`.
- **Champs nullable dans `User`** : l'inscription ne demande que le nom, le prénom, l'email et le mot de passe. L'adresse et le téléphone sont complétés plus tard, et deviennent obligatoires dans le formulaire de livraison.
- **Cascade** : `cascade: ['persist']` enregistre les options, images et lignes de commande en même temps que leur parent ; `orphanRemoval` supprime un élément retiré de son parent.

5.5 Migrations

La structure de la base évolue par **migrations Doctrine** : `make:migration` compare les entités à la base et génère le SQL nécessaire, `doctrine:migrations:migrate` l'applique. Chaque changement de structure est ainsi versionné avec le code et rejoué à l'identique sur le serveur de production.

6. Fonctionnalités principales

6.1 Accueil et catalogue

L'accueil affiche les 3 derniers produits ajoutés et 3 produits à la une, grâce à deux méthodes du repository :

```
public function findLatest(int $limit = 3): array
{
    return $this->findBy([], ['createdAt' => 'DESC'], $limit);
}

public function findFeatured(int $limit = 3): array
{
    return $this->findBy(['featured' => true], ['createdAt' => 'DESC'], $limit);
}
```

Pour garantir qu'il y ait au plus 3 produits à la une, j'ai créé une **contrainte de validation personnalisée** `#[MaxFeatured(max: 3)]` sur l'entité `Product`. Son validateur compte les produits déjà à la une et bloque l'enregistrement d'un quatrième dans le back-office.

Le catalogue propose une recherche par mot-clé, un filtre par catégorie, une fourchette de prix et un tri. Les critères passent dans l'URL (`/catalogue?q=drone&categorie=drones&max=500`), ce qui rend une recherche partageable. La requête est construite avec le QueryBuilder de Doctrine :

```
if ($q) {
    $qb->andWhere('p.name LIKE :q OR p.description LIKE :q')
        ->setParameter('q', '%'.$q.'%');
}
if ($categorySlug) {
    $qb->andWhere('c.slug = :slug')->setParameter('slug', $categorySlug);
}
```

Les valeurs sont toujours passées par `setParameter()`, jamais collées dans la requête, ce qui empêche l'injection SQL. Les images et la catégorie sont chargées dans la même requête (`leftJoin` + `addSelect`) pour éviter une requête supplémentaire par produit.

6.2 Fiche produit et options

L'URL d'un produit utilise son **slug** (`/produit/ecouteurs-pulse-air`), plus lisible et mieux référencé qu'un numéro. Symfony charge le produit automatiquement :

```
#[Route('/produit/{slug}', name: 'app_product_show')]
public function show(#[MapEntity(mapping: ['slug' => 'slug'])] Product $product): Response
```

Une image peut être liée à une option. Quand le client choisit « Couleur : Blanc », la photo principale change ; inversement, cliquer sur la miniature blanche sélectionne l'option « Blanc ». Chaque option de la liste porte l'adresse de son image dans un attribut `data-image`, lu par le JavaScript.

6.3 Panier en session et en AJAX

Toute la logique du panier est regroupée dans le service `CartService`. En session, il ne stocke que des identifiants :

```
public function add(Product $product, ?ProductOption $option = null): void
{
    $lines = $this->getRawLines();
    $lines[] = [
        'id' => bin2hex(random_bytes(6)), // identifiant unique de la ligne
        'product' => $product->getId(),
        'option' => $option?->getId(),
    ];
    $this->save($lines);
}
```

Les produits sont rechargés depuis la base à chaque affichage : le prix est toujours à jour, et un produit supprimé disparaît du panier.

Côté navigateur, `assets/cart.js` intercepte l'envoi des formulaires d'ajout et de suppression, les envoie avec `fetch()` et met à jour le compteur et le total sans recharger la page :

```
const response = await fetch(form.action, {
    method: 'POST',
    body: new FormData(form),
    headers: { Accept: 'application/json', 'X-Requested-With': 'XMLHttpRequest' },
});
const data = await response.json();
if (data.success) {
    updateCount(data.count);
}
```

Le contrôleur répond en JSON pour un appel AJAX, et par une redirection classique sinon : le site reste utilisable sans JavaScript. Il vérifie aussi le jeton CSRF et que l'option choisie appartient bien au produit.

6.4 Authentification

- Inscription (`make:registration-form`) : prénom, nom, email unique, mot de passe de 8 caractères minimum, acceptation des conditions générales. Le client est connecté automatiquement après l'inscription.
- Connexion par formulaire (`make:security:form-login`) avec option « Se souvenir de moi ».
- Rôles : chaque utilisateur a `ROLE_USER` ; les administrateurs ont en plus `ROLE_ADMIN`.

Le formulaire d'inscription a évolué pendant le projet. La première version générée par `make:registration-form` ne demandait que l'email et le mot de passe. J'ai ensuite ajouté le prénom et le nom, obligatoires, avant de mettre en forme la page avec Bootstrap.

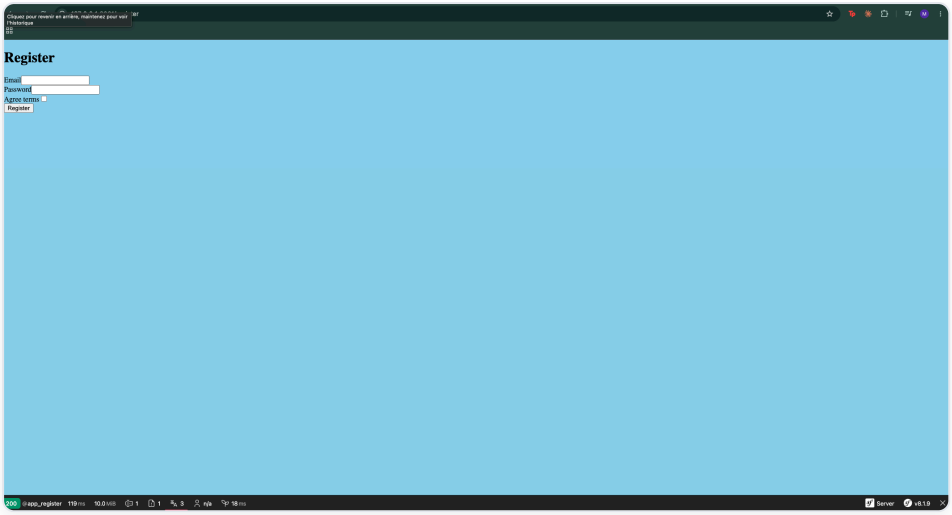


Figure — Première version du formulaire généré (email et mot de passe seulement).

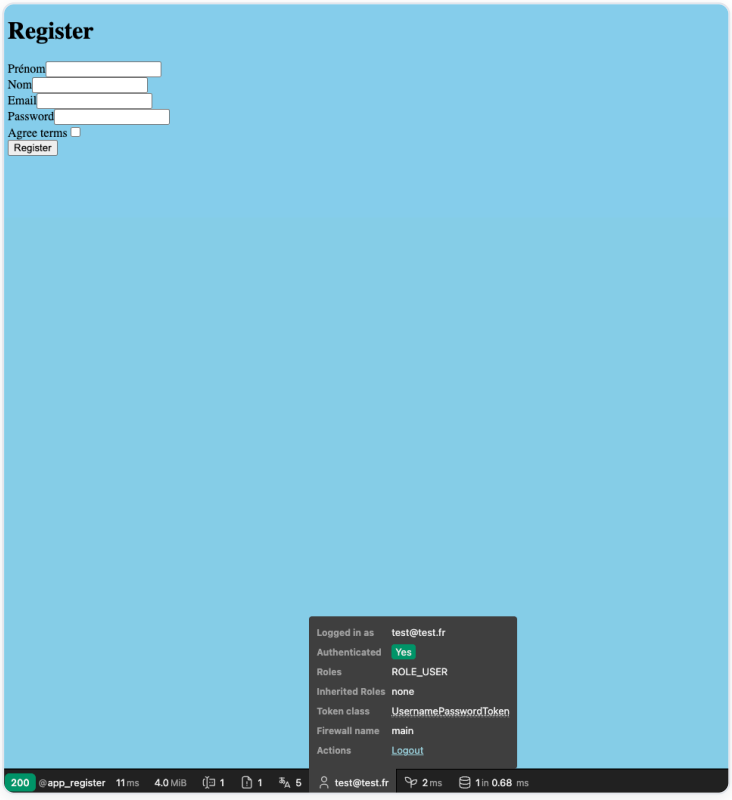


Figure — Prénom et nom ajoutés ; la barre du profiler confirme la connexion avec le rôle ROLE_USER.

6.5 Processus de commande

La commande se déroule en trois étapes, signalées par un indicateur visuel, et n'est accessible qu'aux utilisateurs connectés :

1. **Panier** : vérification des articles et du total.
2. **Livraison** : formulaire pré-rempli avec l'adresse du profil, validé par des contraintes (champs obligatoires, code postal à 5 chiffres, numéro de téléphone français). Une case permet d'enregistrer l'adresse dans le profil.
3. **Confirmation** : récapitulatif complet puis validation.

À la validation, le contrôleur crée la commande à partir du panier :

```
foreach ($this->cart->getLines() as $line) {
    $product = $line['product'];
    $order->addOrderLine((new OrderLine())
        ->setProduct($product)
        ->setProductOption($line['option'])
        ->setProductName($product->getName())
        ->setPrice($product->getPrice()));
    $total += $product->getPrice();
}
$order->setTotal($total);
```

Le total est recalculé côté serveur : on ne fait jamais confiance à un montant envoyé par le navigateur. Le panier est ensuite vidé, l'email récapitulatif envoyé, et le client redirigé vers une page de remerciement.

6.6 Emails

Le service `OrderMailer` envoie deux emails, mis en page avec des templates Twig :

- le **récapitulatif de commande**, juste après la validation ;
- une **notification de changement de statut** (validée, expédiée, annulée).

Le second est déclenché par un **listener Doctrine** : `preUpdate` détecte que le champ `status` a changé, puis `postUpdate` envoie l'email une fois la modification enregistrée. Ainsi, quelle que soit la façon dont le statut change, le client est prévenu, sans code ajouté dans le back-office. L'envoi est placé dans un `try/catch` : une panne du serveur d'emails ne fait jamais échouer une commande.

6.7 Espace client

Page	Contenu
Tableau de bord	Nombre de commandes, dépenses du mois comparées à l'objectif fixé (barre de progression), 3 dernières commandes, informations personnelles
Mes commandes	Historique avec un badge de couleur par statut
Détail d'une commande	Suivi visuel (En attente, Validée, Expédiée), articles, adresse, facture PDF
Profil	Coordonnées, adresse, objectif de dépenses mensuel
Mot de passe	Vérification de l'ancien mot de passe, nouveau mot de passe saisi deux fois

La facture PDF est générée avec **Dompdf** à partir d'un template Twig. L'objectif de dépenses répond au cahier des charges (« suivre les objectifs d'épargne ou de dépenses ») : une requête additionne les commandes du mois, hors commandes annulées.

6.8 Back-office avec EasyAdmin

Écran	Fonctionnalités
Tableau de bord	Ventes des 14 derniers jours (graphique), commandes en attente, nombre de produits et de clients, dernières commandes
Produits	Création, modification, suppression ; prix saisi en euros et stocké en centimes ; options ajoutées dans le formulaire du produit
Images	Upload, ordre d'affichage, lien facultatif avec une option
Catégories	Gestion avec slug automatique
Commandes	Liste avec badges de statut et filtres (statut, date, client), détail complet ; seul le statut est modifiable
Clients	Coordonnées, attribution du rôle administrateur, historique des commandes

Les commandes sont créées par les clients : la création et la suppression sont désactivées dans le back-office pour éviter les erreurs de manipulation.

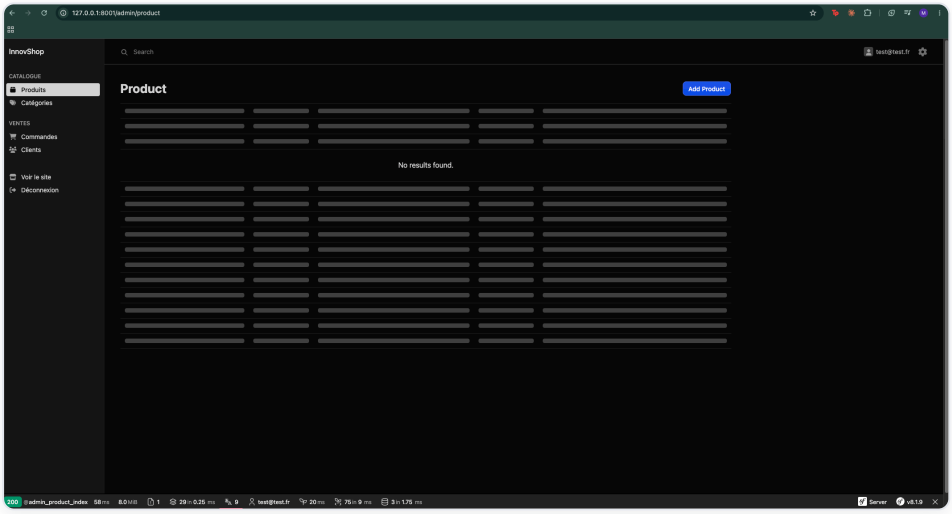


Figure — Première version du back-office juste après `make:admin:crud` : la base est vide et les libellés sont encore en anglais. Les captures 14 à 20 montrent la version finale.

7. Sécurité et bonnes pratiques

7.1 Sécurité

Risque	Protection mise en place
Vol de mots de passe	Mots de passe hachés avec l'algorithme `auto` de Symfony (bcrypt ou argon2), jamais stockés en clair
Injection SQL	Requêtes Doctrine et paramètres liés (`setParameter()`)
Faille XSS	Échappement automatique de toutes les variables par Twig
Faille CSRF	Jetons CSRF sur la connexion, l'inscription, les formulaires, l'ajout et la suppression au panier, la validation de commande
Accès non autorisé	`access_control` dans `security.yaml` et attributs `#[IsGranted]` sur `/admin`, `/compte` et `/commande`
Consultation de la commande d'un autre client	Voter `OrderVoter` : seul le propriétaire ou un administrateur y accède, sinon erreur 403
Données invalides	Contraintes `Assert` sur les entités et dans les formulaires
Montant ou option falsifiés	Le serveur recalcule le total et vérifie que l'option appartient au produit
Fuite de secrets	Identifiants dans `.env.local`, exclu du dépôt Git
Fichiers envoyés	Images renommées avec un nom aléatoire à l'upload

Le back-office est protégé à deux niveaux :

```yaml
# config/packages/security.yaml
access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
    - { path: ^/compte, roles: ROLE_USER }
    - { path: ^/commande, roles: ROLE_USER }
```

```
#[AdminDashboard(routePath: '/admin', routeName: 'admin')]
#[IsGranted('ROLE_ADMIN')]
class DashboardController extends AbstractDashboardController
```

Le voter décide de l'accès à une commande précise :

```
protected function voteOnAttribute(string $attribute, mixed $subject, TokenInterface $token, ?Vote
$vote = null): bool
{
    $user = $token->getUser();
    if (!$user instanceof User) {
        return false;
    }
    if (\in_array('ROLE_ADMIN', $user->getRoles(), true)) {
        return true;
    }

    return $subject->getUser() === $user;
}
```

Des pages d'erreur 404 et 403 personnalisées remplacent les pages par défaut de Symfony en production.

7.2 Bonnes pratiques Symfony appliquées

1. **Logique métier dans des services** (`CartService`, `OrderMailer`) : les contrôleurs restent courts et lisibles.
2. **Validation par attributs** `Assert` directement sur les entités, et une contrainte personnalisée (`MaxFeatured`).
3. **Secrets hors du dépôt** : `.env.local` n'est jamais versionné.
4. **Configuration par attributs PHP** : `#[Route]`, `#[IsGranted]`, `#[ORM\...]`, `#[MapEntity]`.
5. **Voter** pour les droits dépendant d'un objet.
6. **Énumération PHP** pour les statuts.
7. **Requêtes dans les repositories**, jamais dans les contrôleurs ni les templates.
8. **Templates découpés** : un layout commun, des partials réutilisables, une extension Twig.
9. **Listener Doctrine** pour réagir à un événement sans coupler le code au back-office.
10. **AssetMapper**, choix par défaut des versions récentes de Symfony.

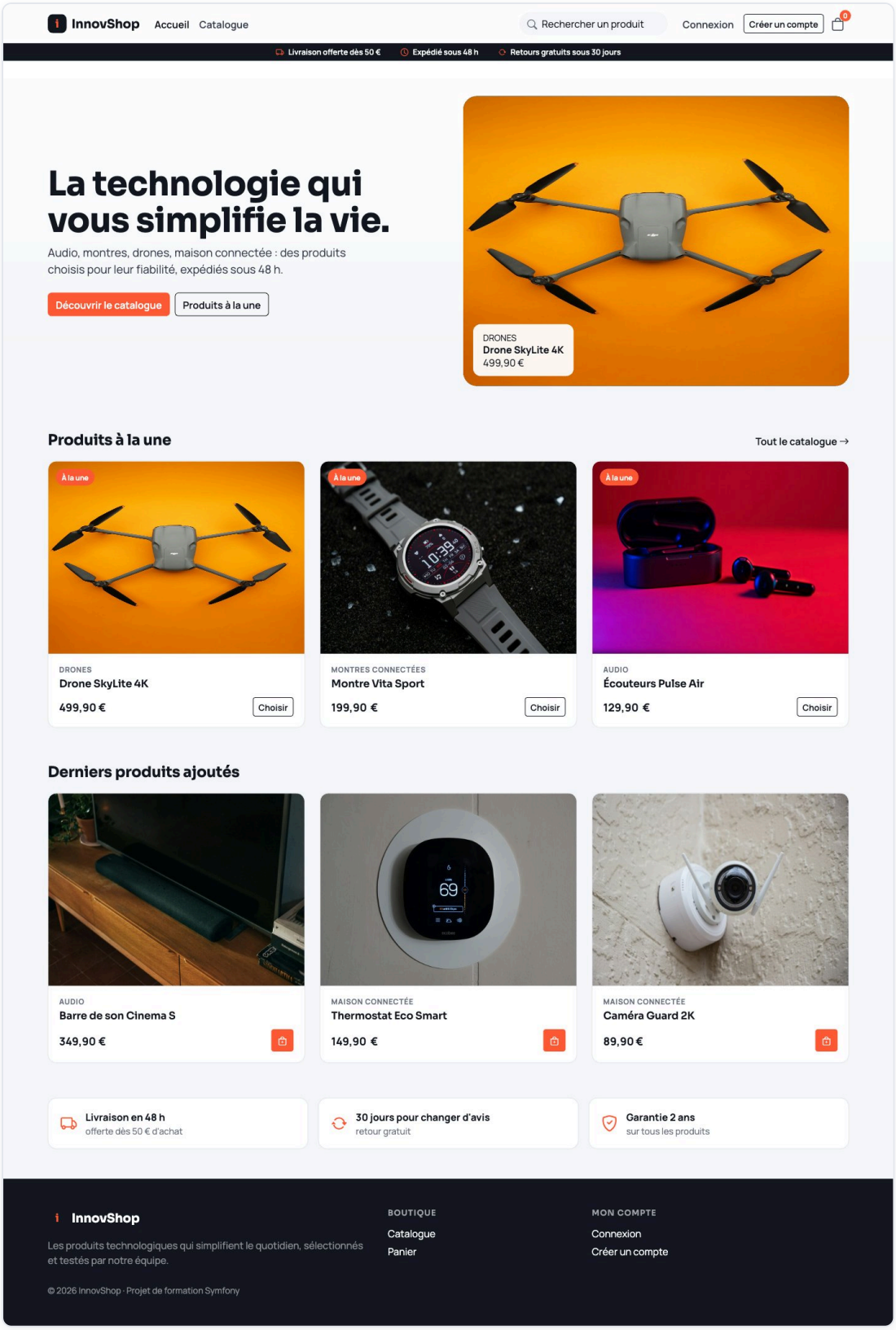
7.3 Accessibilité et responsive

- Mise en page Bootstrap adaptée au mobile, à la tablette et à l'ordinateur.
- Labels associés à tous les champs, textes alternatifs sur les images, `aria-label` sur les boutons qui ne contiennent qu'une icône.
- Focus visible au clavier, animations désactivées si l'utilisateur préfère réduire les mouvements (`prefers-reduced-motion`).

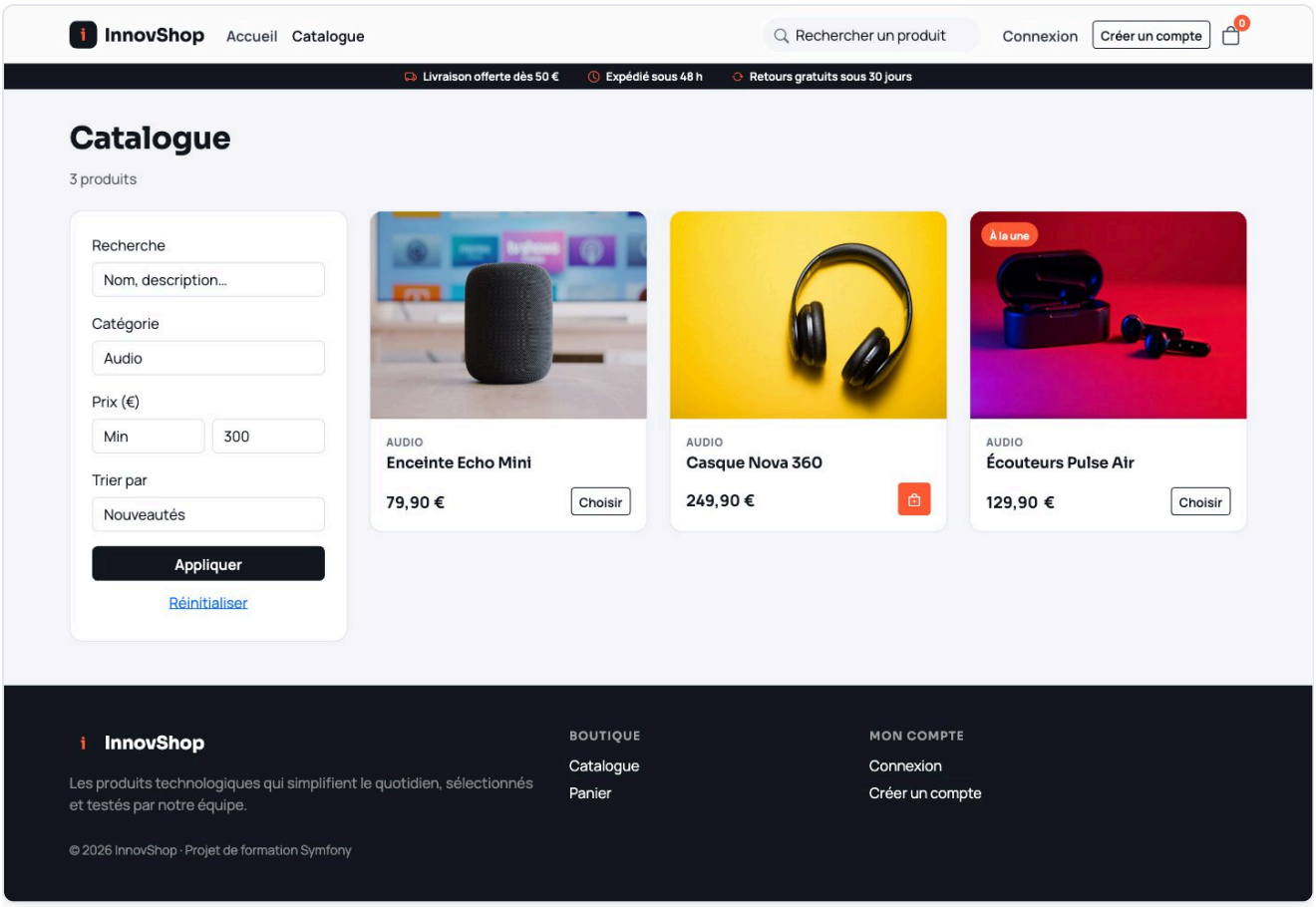
8. Démonstration visuelle

Comptes de démonstration : `client@innovshop.fr` / `Client1234!` (client) et `admin@innovshop.fr` / `Admin1234!` (administrateur).

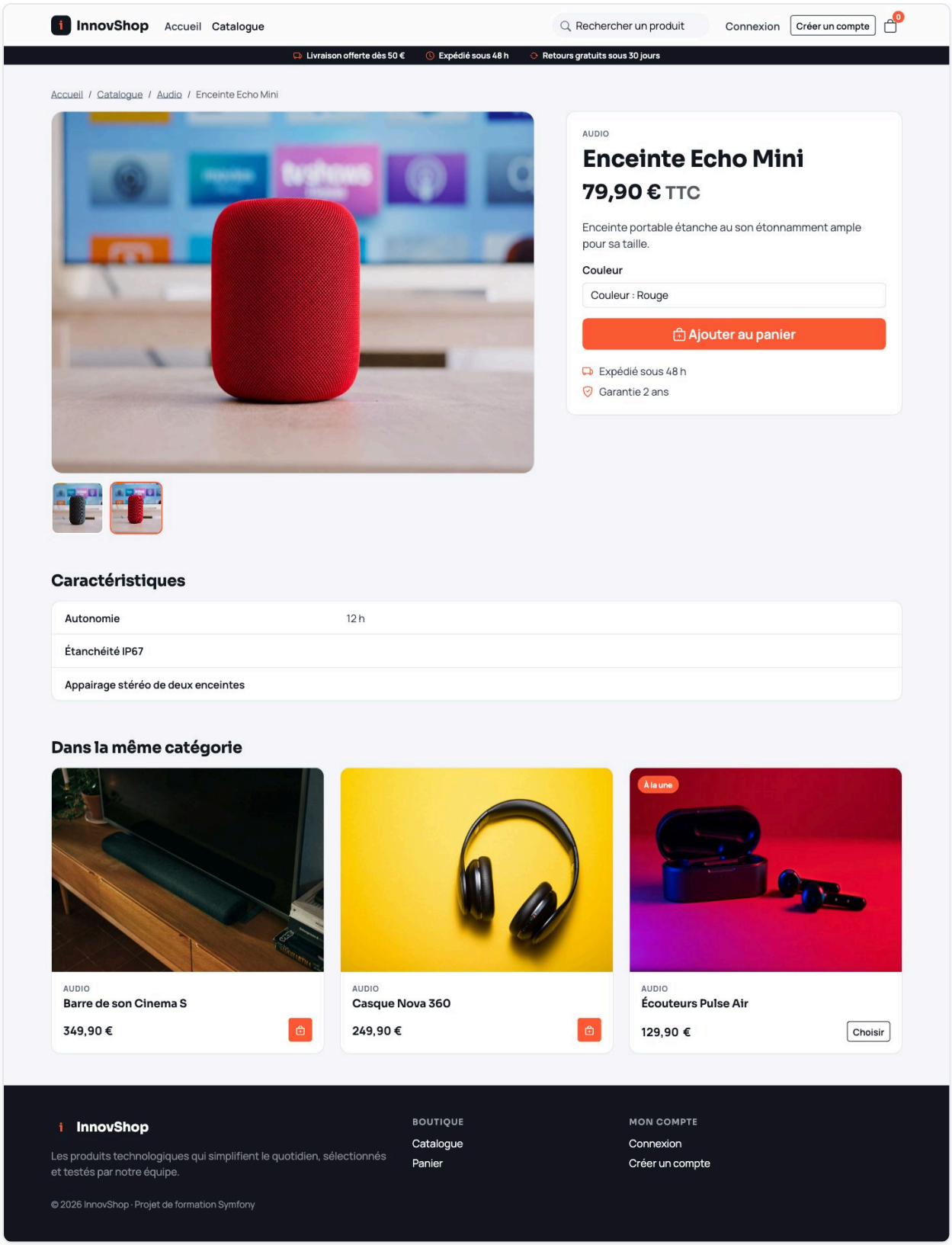
8.1 Parcours client



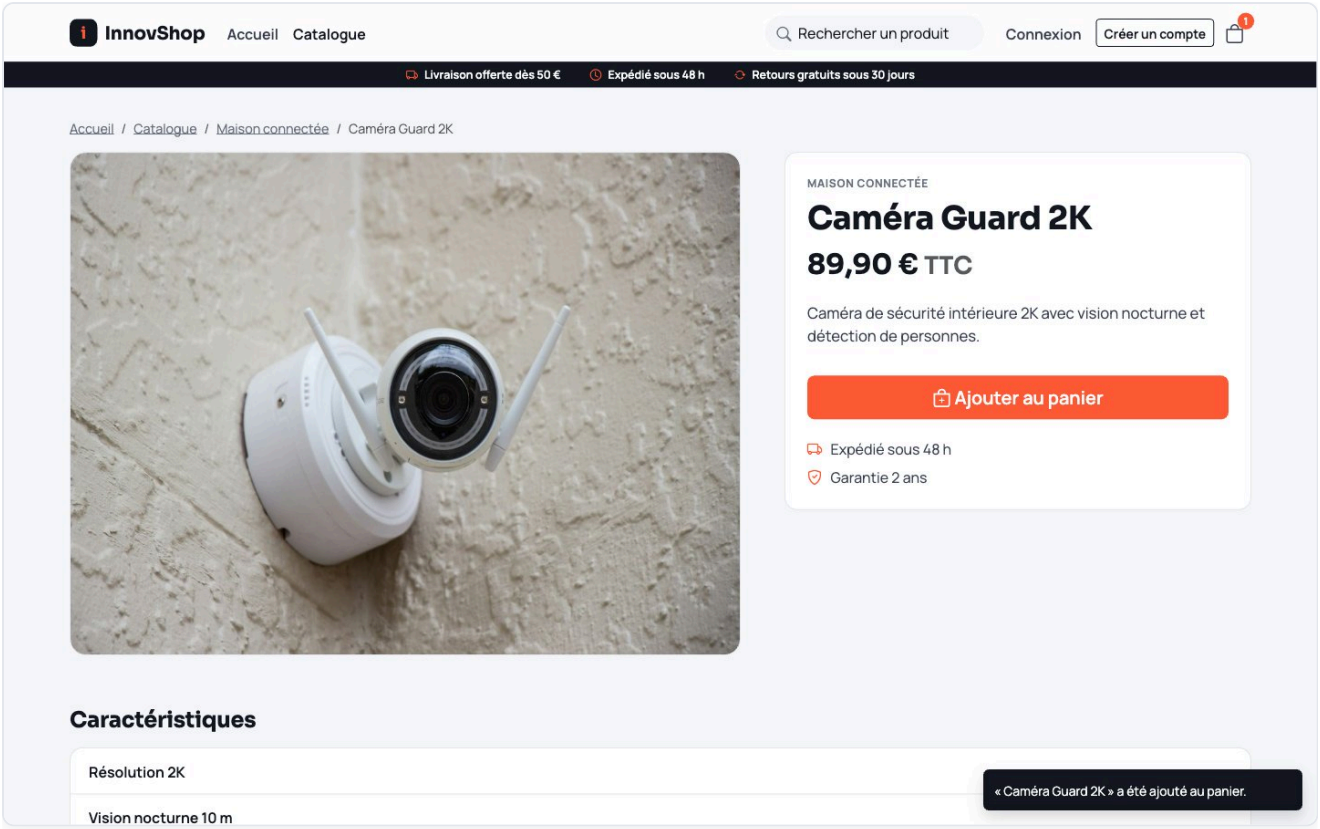
Capture 1 — Page d'accueil : bandeau de livraison, visuel principal, produits à la une et derniers produits ajoutés.



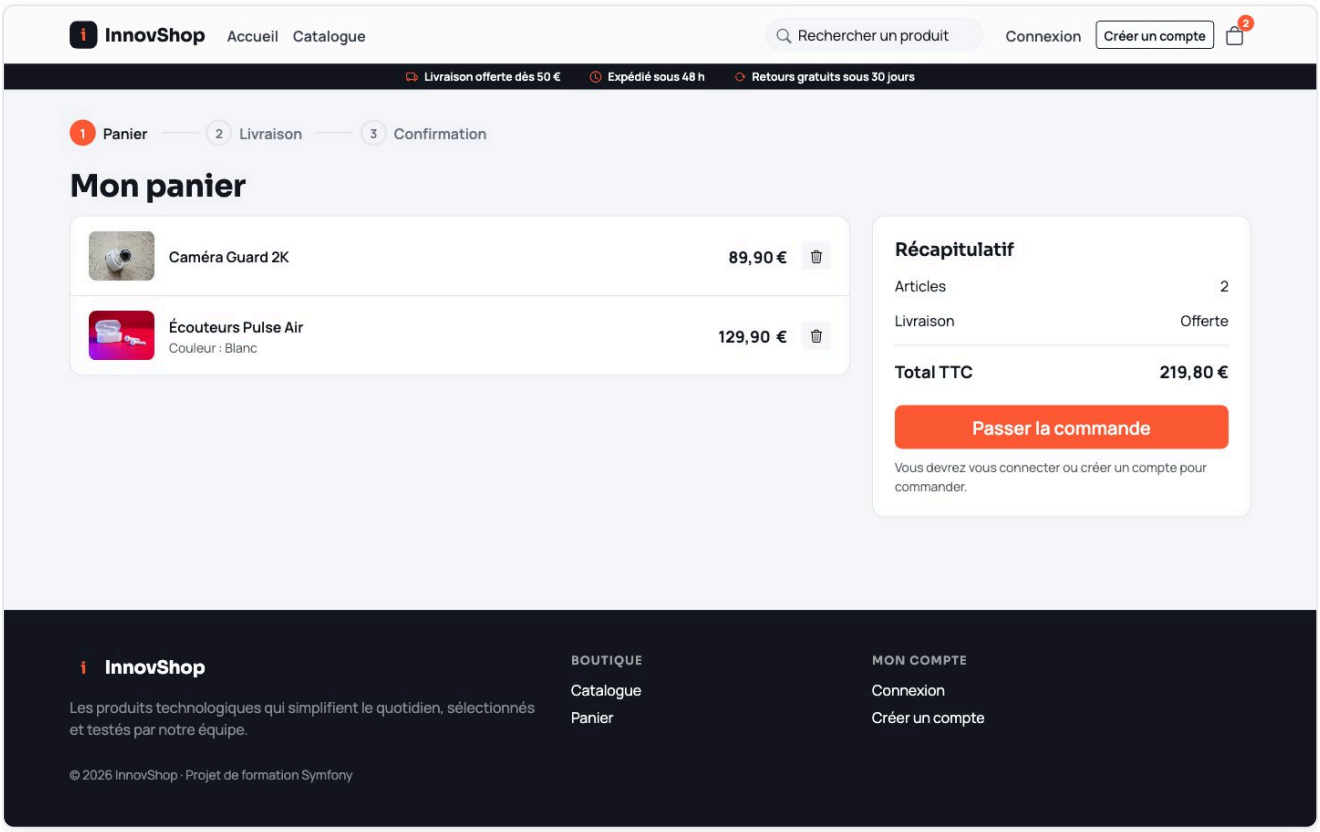
Capture 2 — Catalogue avec un filtre appliqué (catégorie, fourchette de prix, tri).



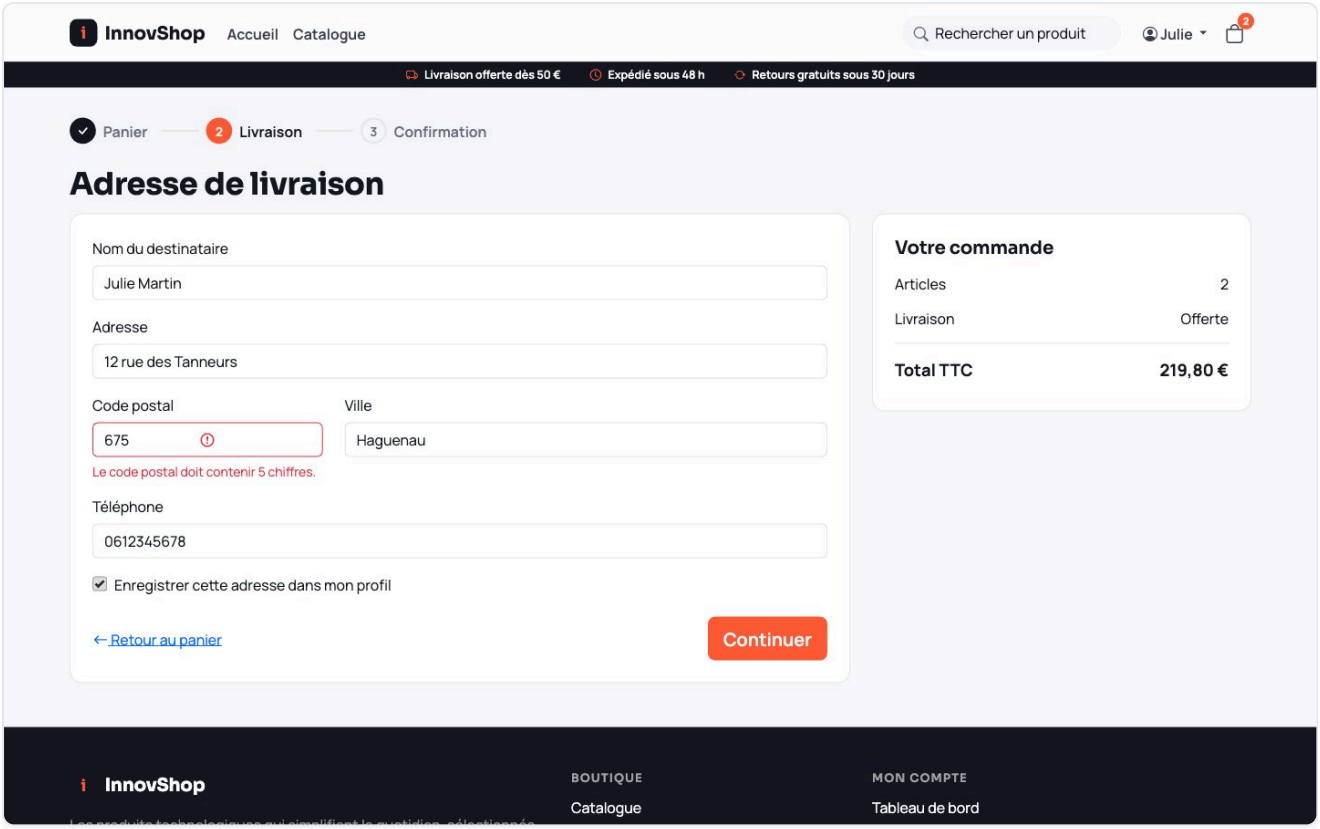
Capture 3 — Fiche produit Enceinte Echo Mini : l'option Rouge est sélectionnée et la photo rouge s'affiche.



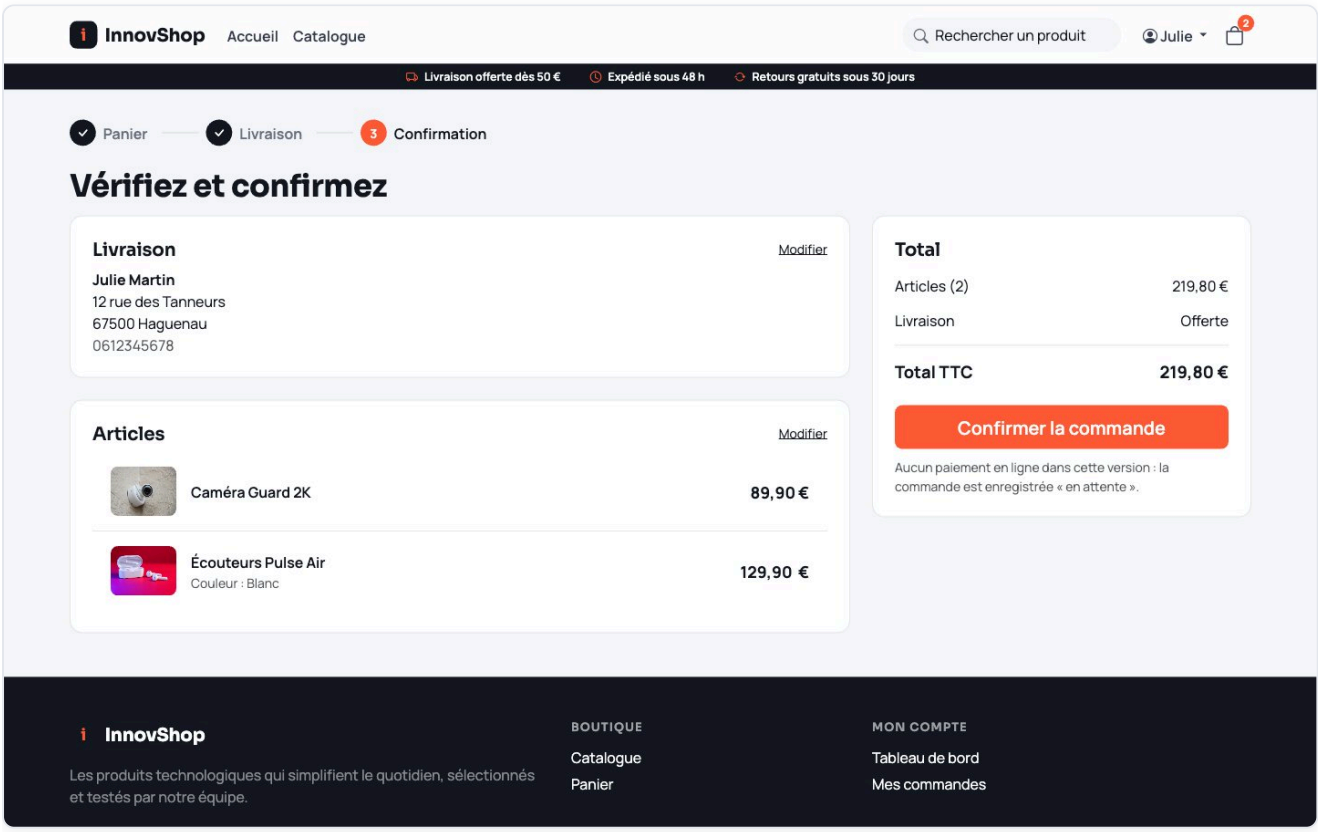
Capture 4 — Notification « ajouté au panier » et compteur du menu mis à jour sans rechargement.



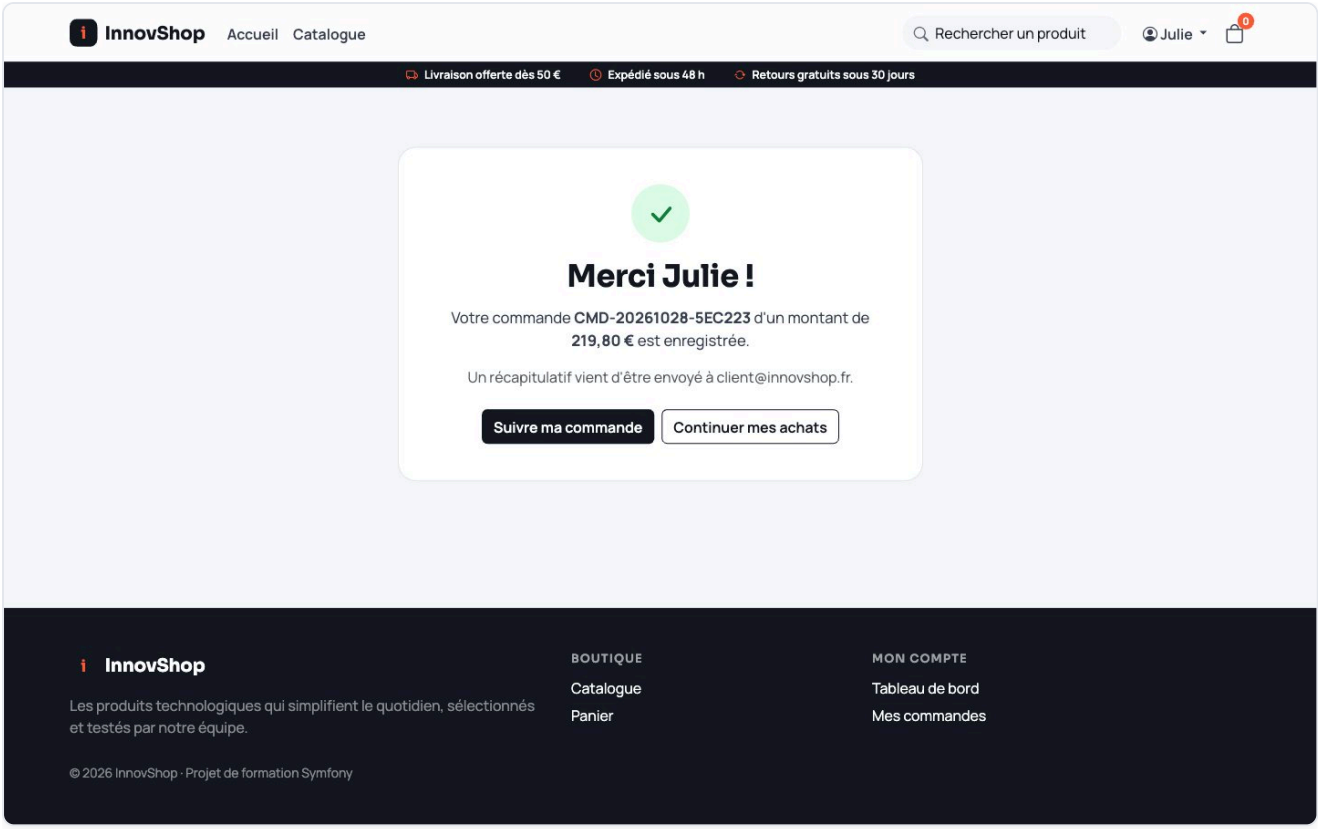
Capture 5 — Page panier (étape 1 du tunnel de commande).



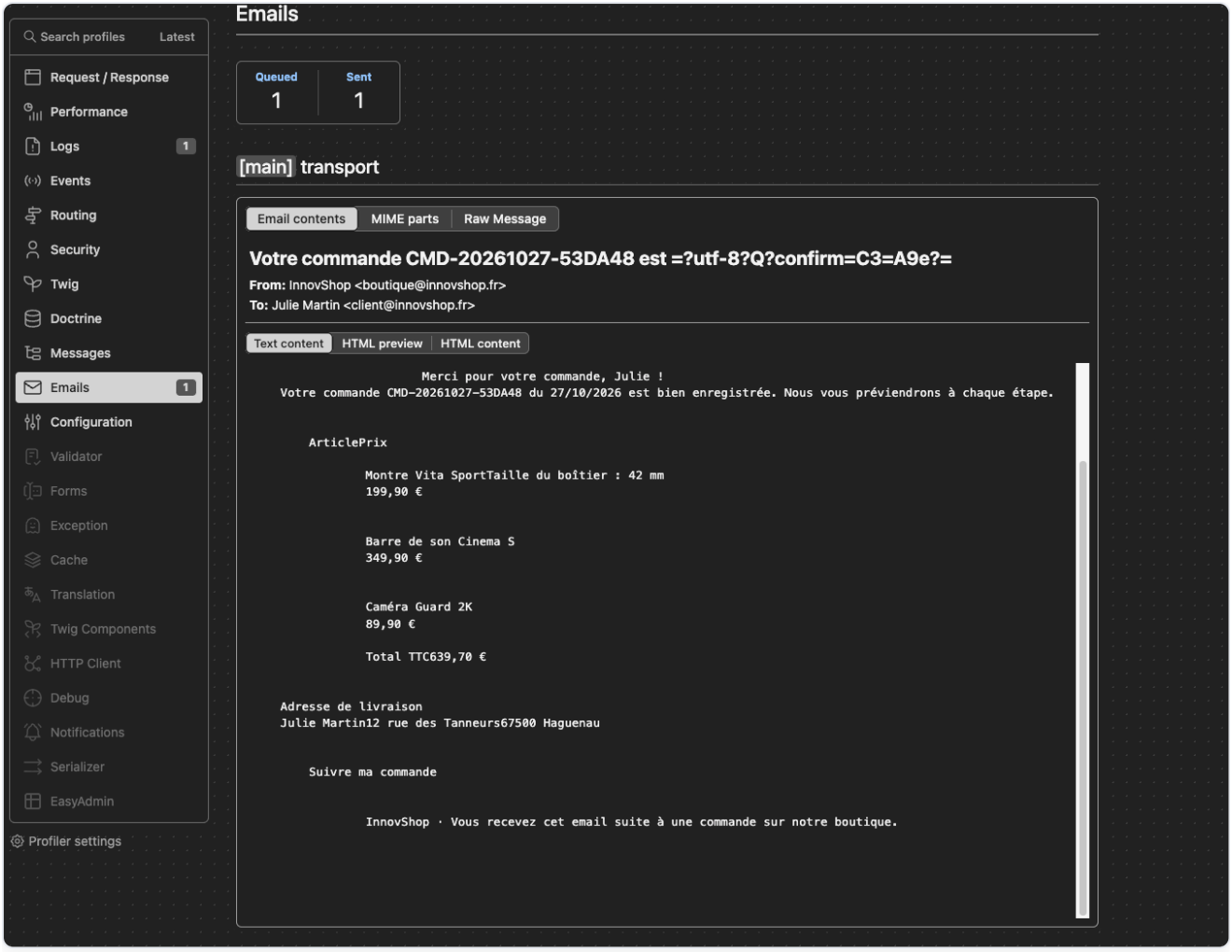
Capture 6 — Formulaire de livraison : le code postal invalide est refusé par la validation serveur.



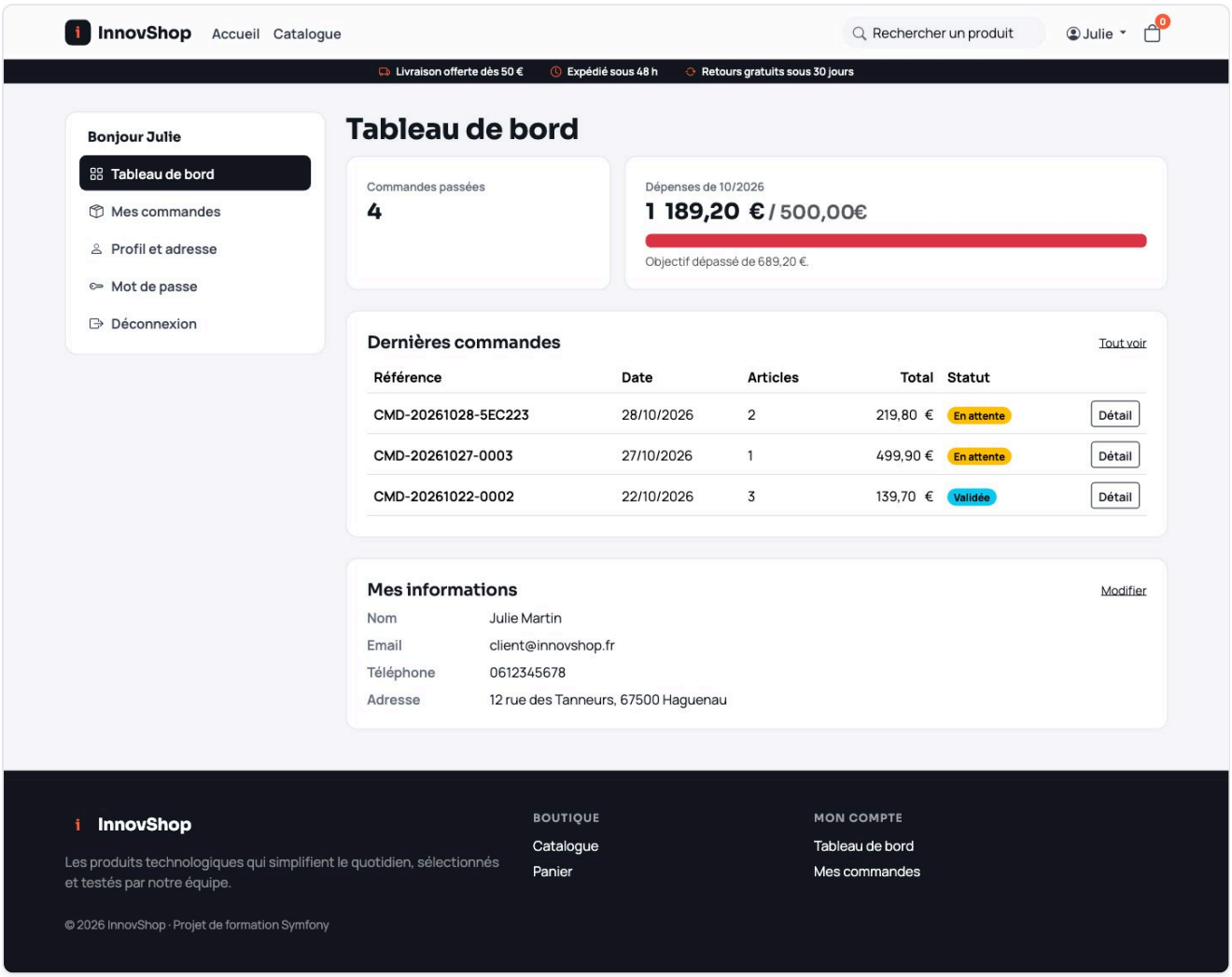
Capture 7 — Page de confirmation (étape 3) : récapitulatif des articles et de l'adresse avant validation.



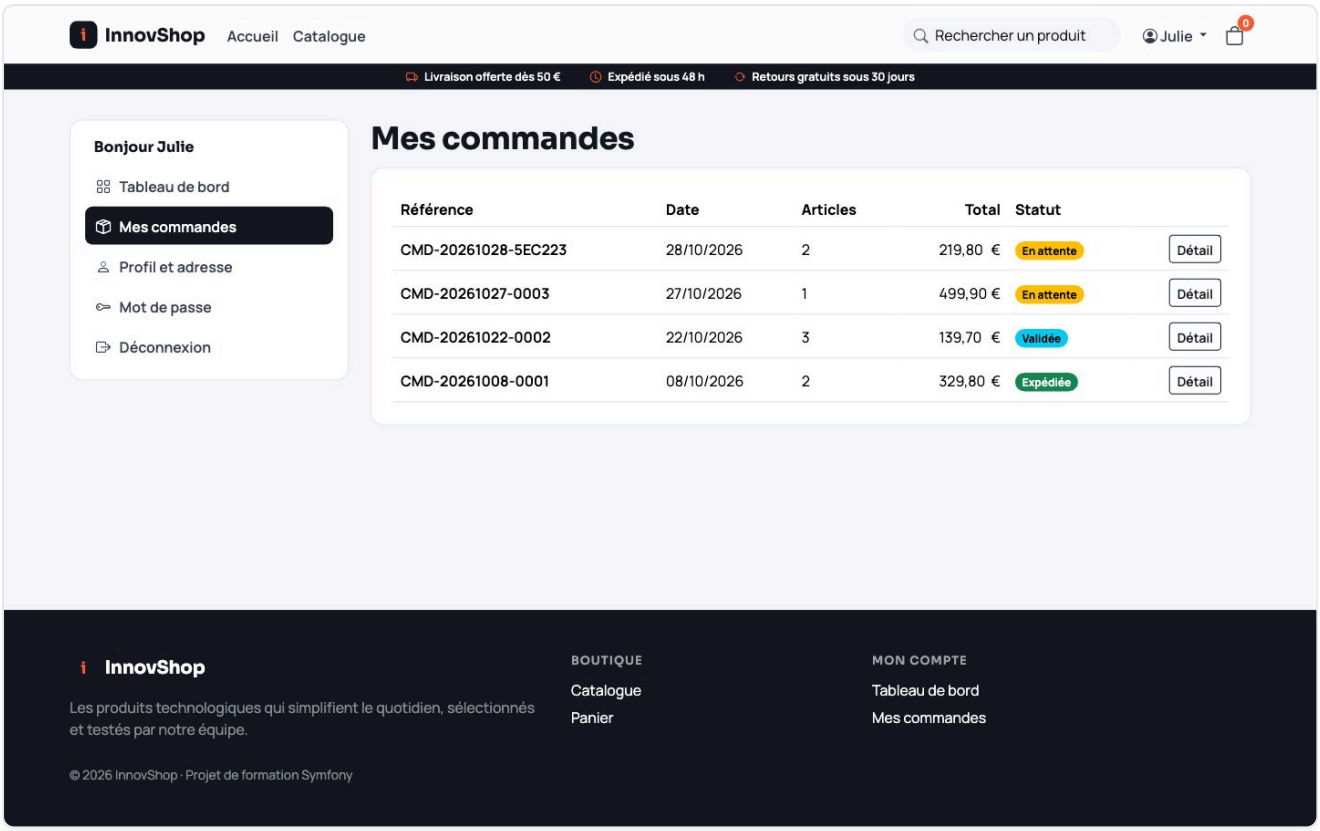
Capture 8 — Page « Merci » avec la référence de la commande.



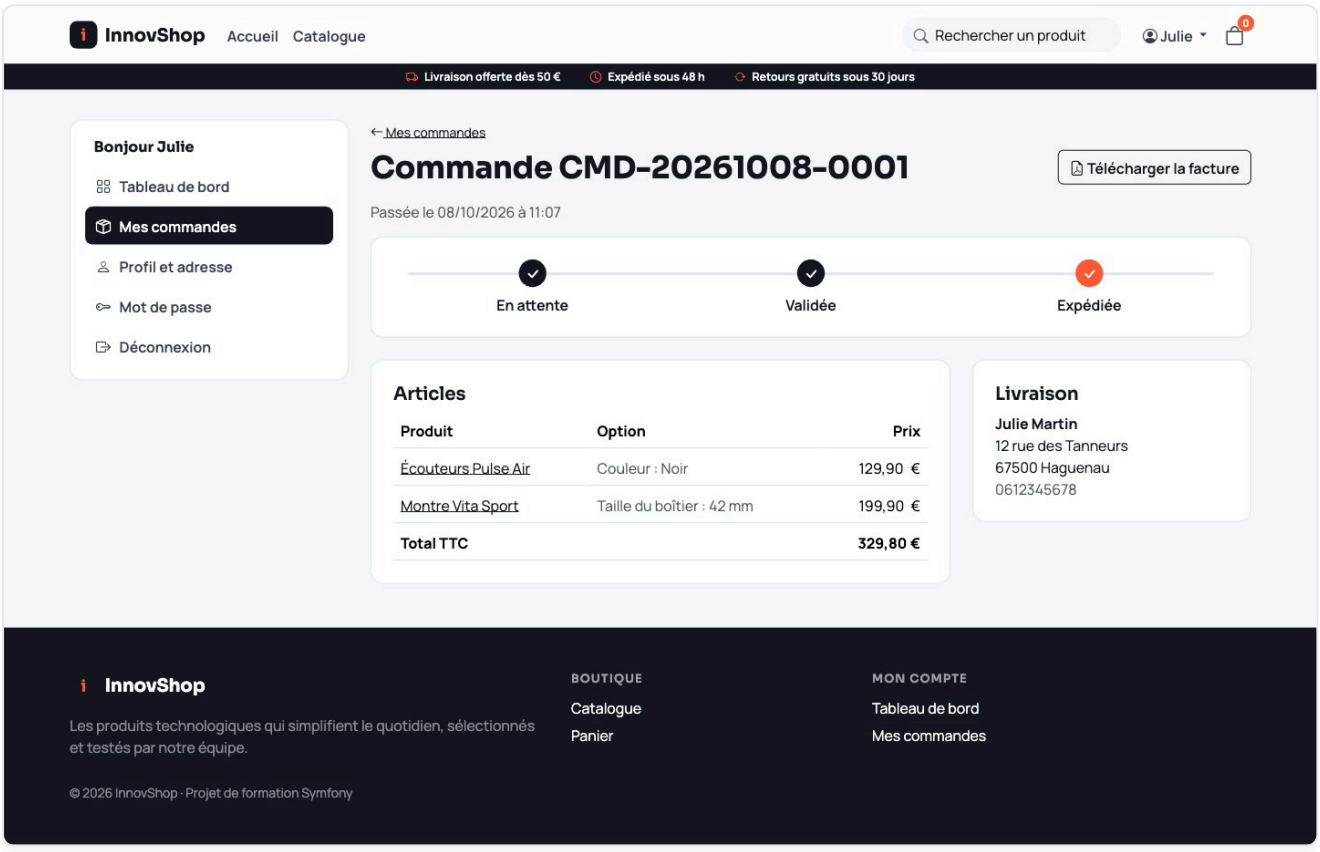
Capture 9 — Email récapitulatif reçu après la commande, vu dans le profiler Symfony (onglet Emails) en local.



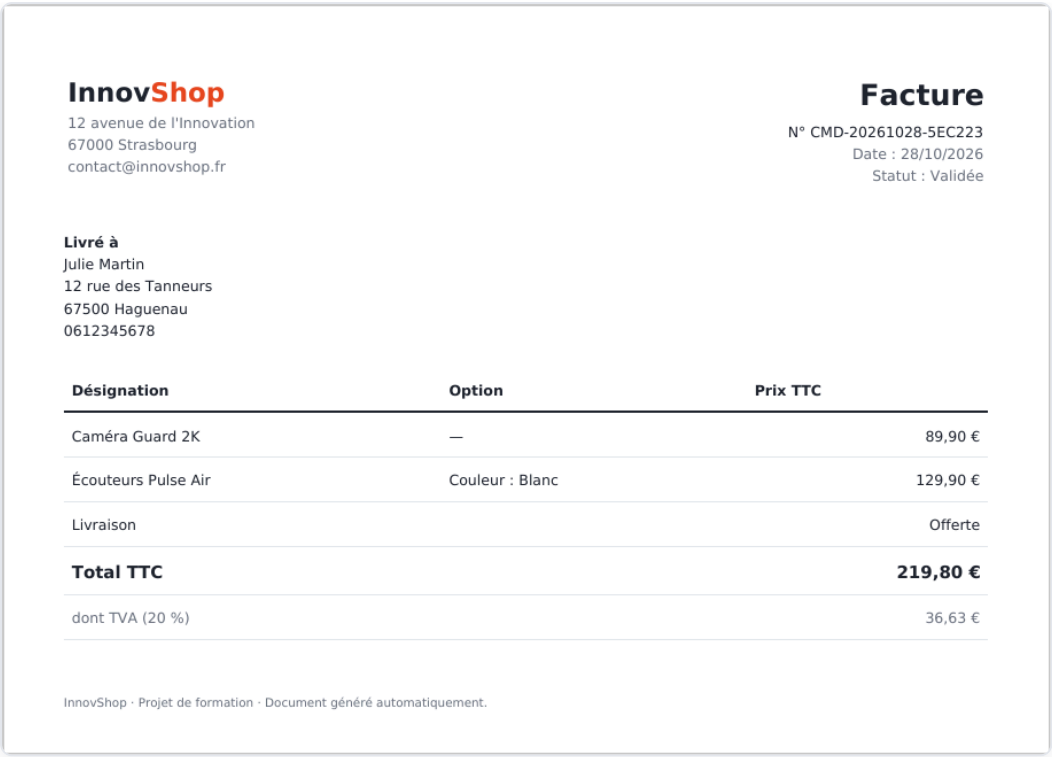
Capture 10 — Espace client : tableau de bord avec l'objectif de dépenses du mois.



Capture 10 bis — Liste « Mes commandes » avec le statut de chaque commande.



Capture 11 — Détail d'une commande avec le suivi visuel des étapes.



InnovShop
12 avenue de l'Innovation
67000 Strasbourg
contact@innovshop.fr

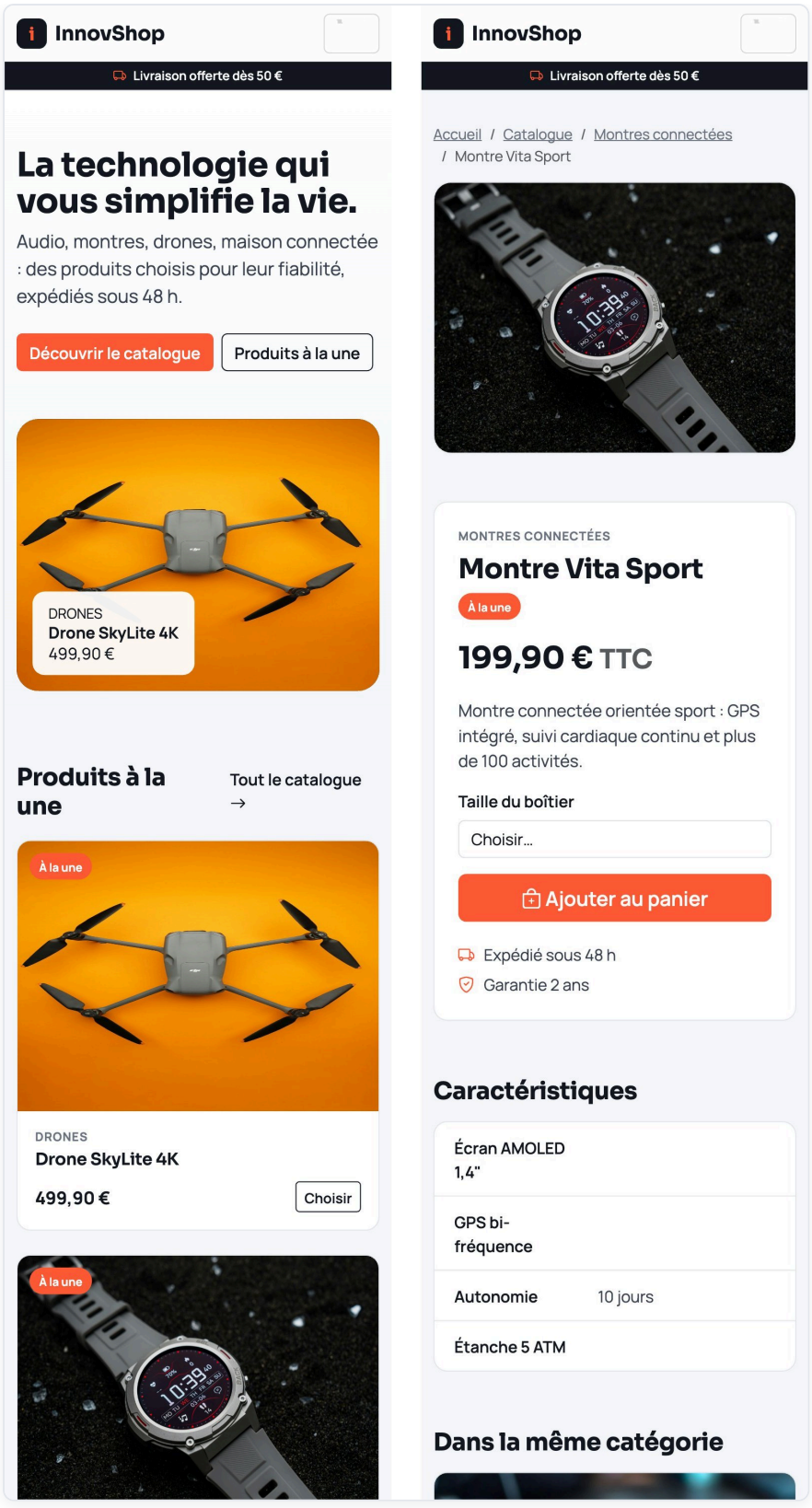
Facture
N° CMD-20261028-5EC223
Date : 28/10/2026
Statut : Validée

Livré à
Julie Martin
12 rue des Tanneurs
67500 Haguenau
0612345678

Désignation	Option	Prix TTC
Caméra Guard 2K	—	89,90 €
Écouteurs Pulse Air	Couleur : Blanc	129,90 €
Livraison		Offerte
Total TTC		**219,80 €**
dont TVA (20 %)		36,63 €

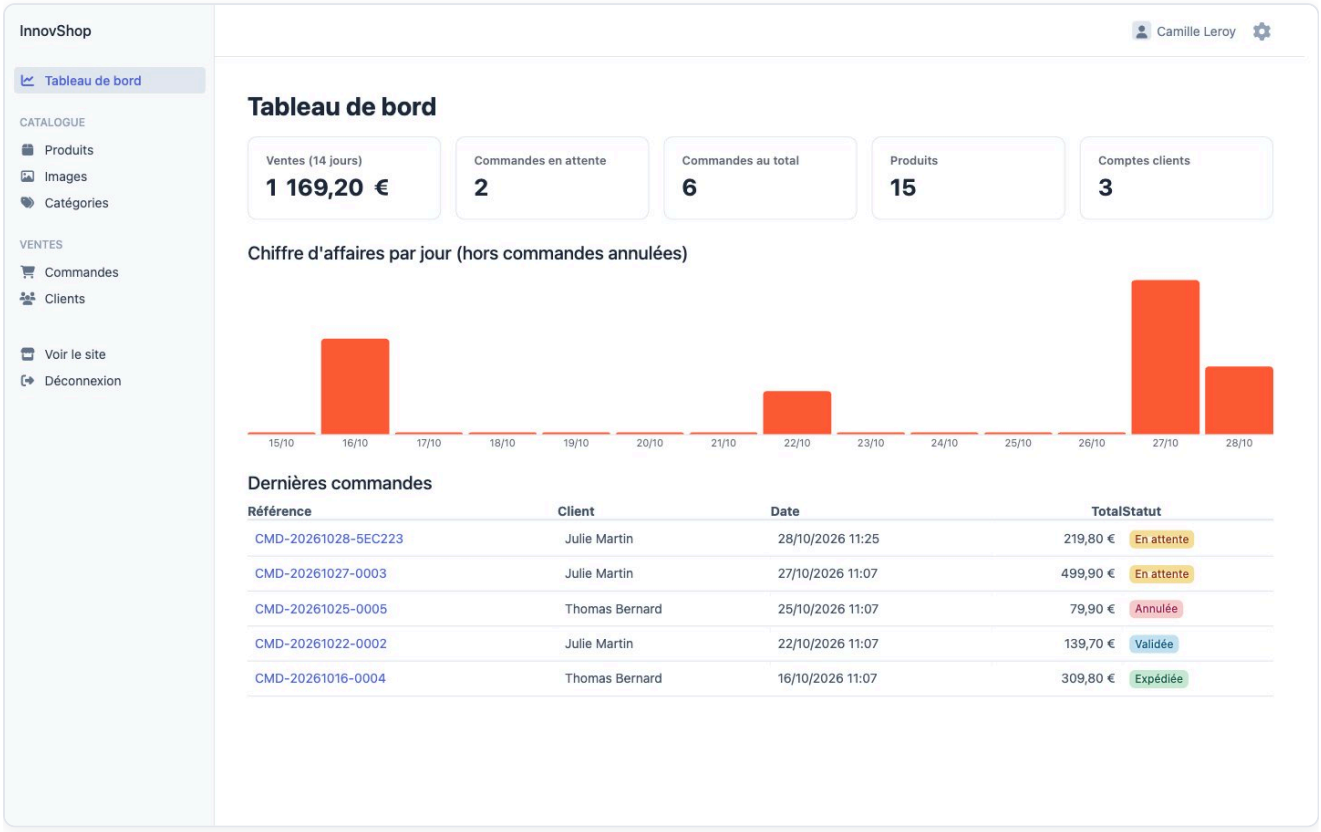
InnovShop · Projet de formation · Document généré automatiquement.

Capture 12 — Facture PDF générée par Dompdf depuis l'espace client : référence, adresse de livraison, lignes avec option, total TTC et TVA.

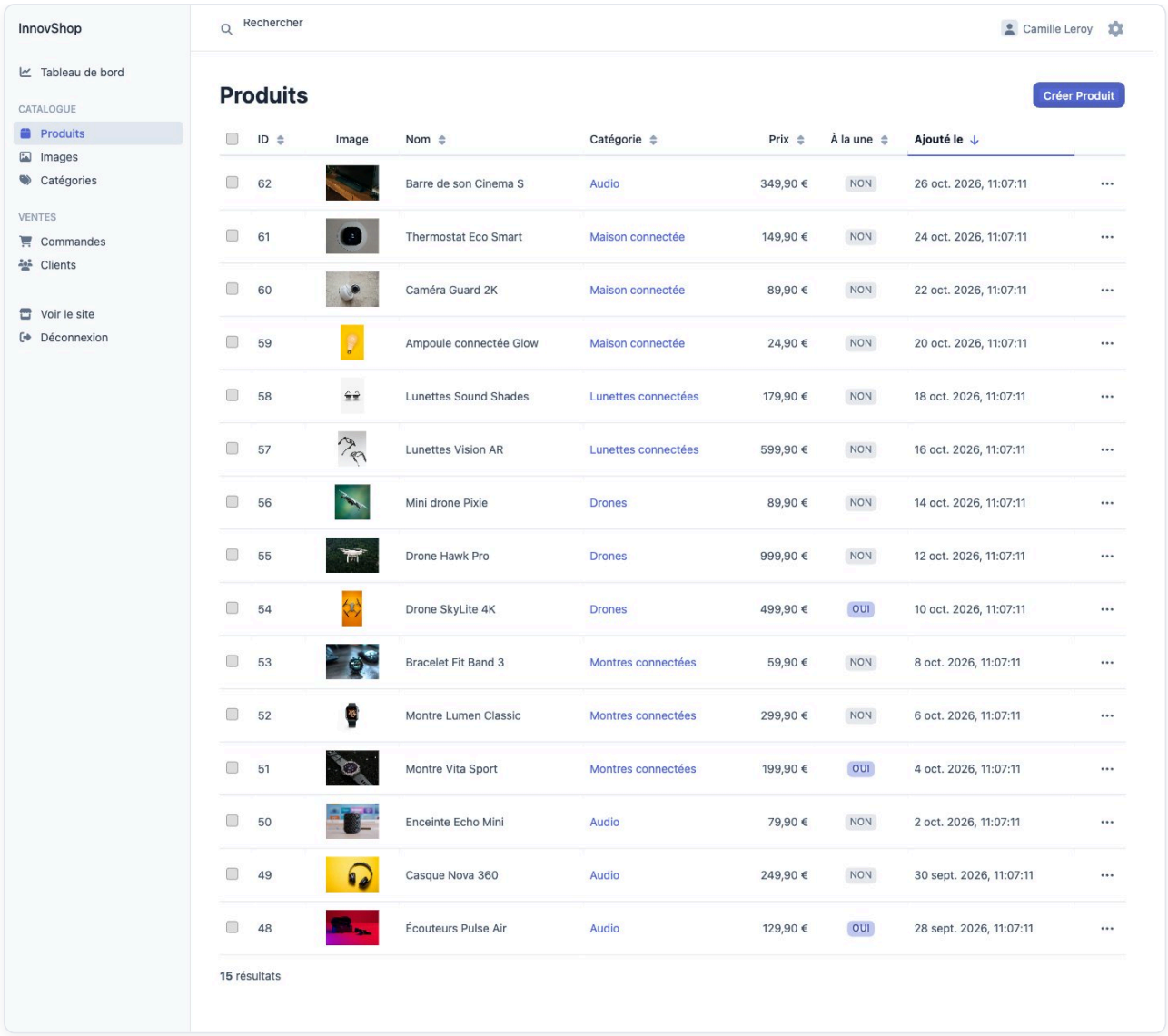


Capture 13 — Version mobile (375 px) : accueil à gauche, fiche produit à droite.

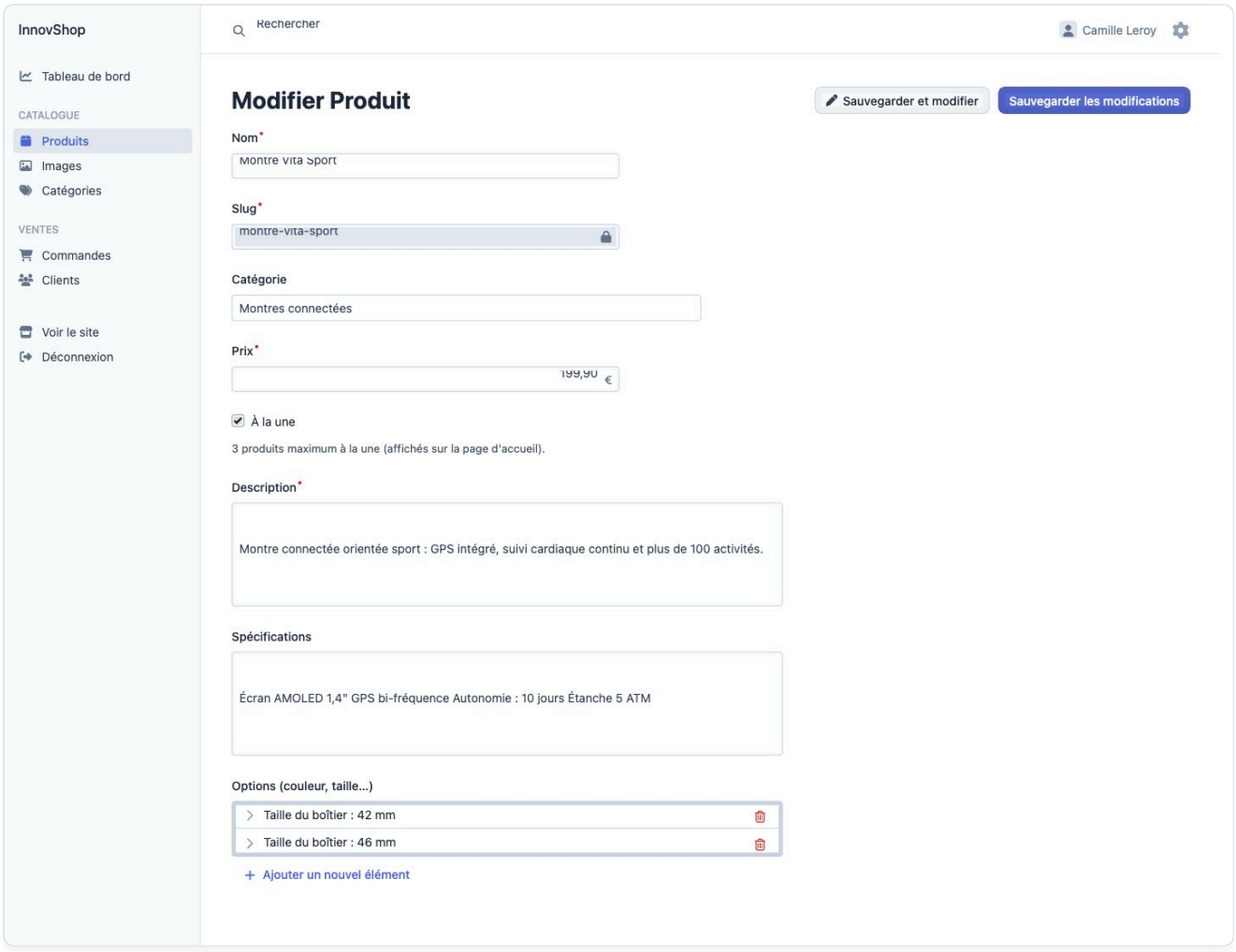
8.2 Parcours administrateur



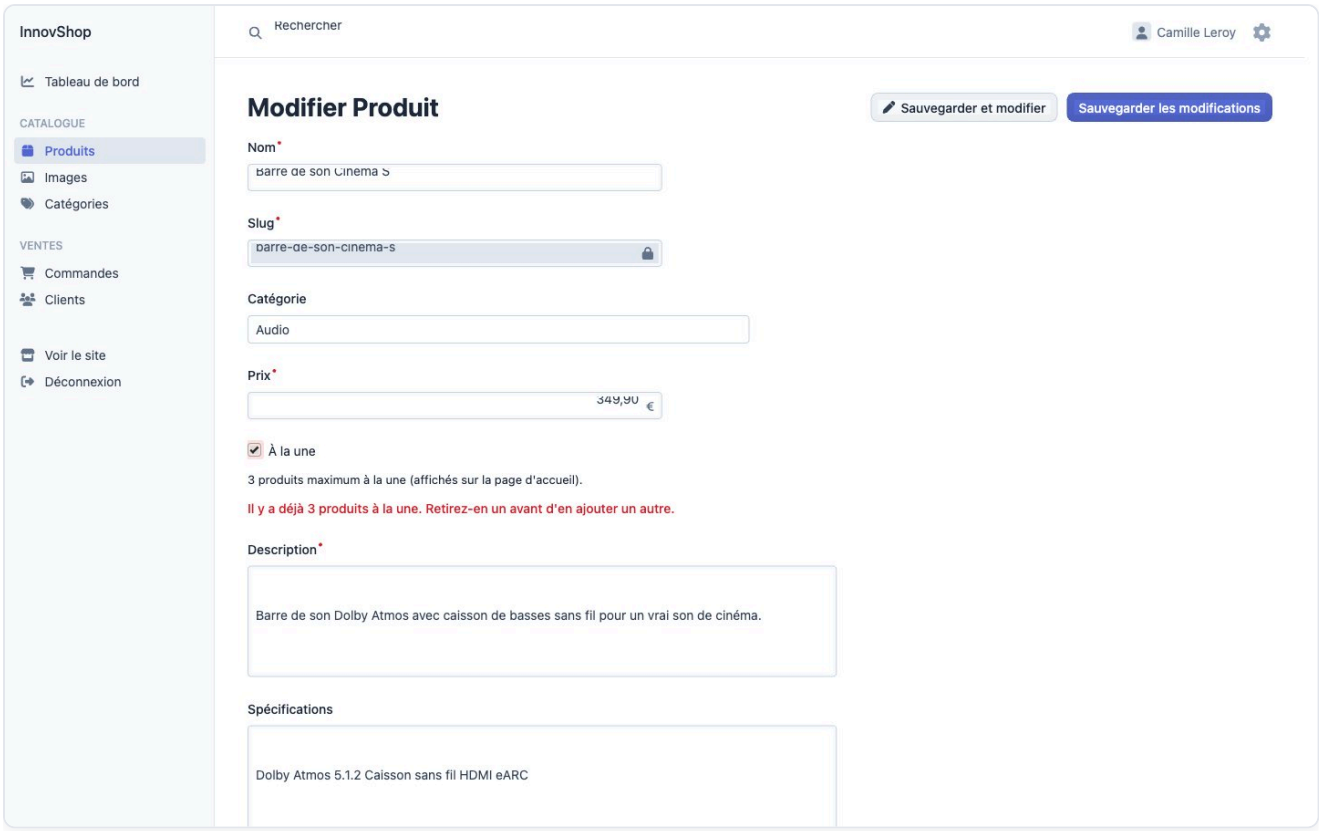
Capture 14 — Tableau de bord admin : chiffres clés, chiffre d'affaires par jour sur 14 jours et dernières commandes.



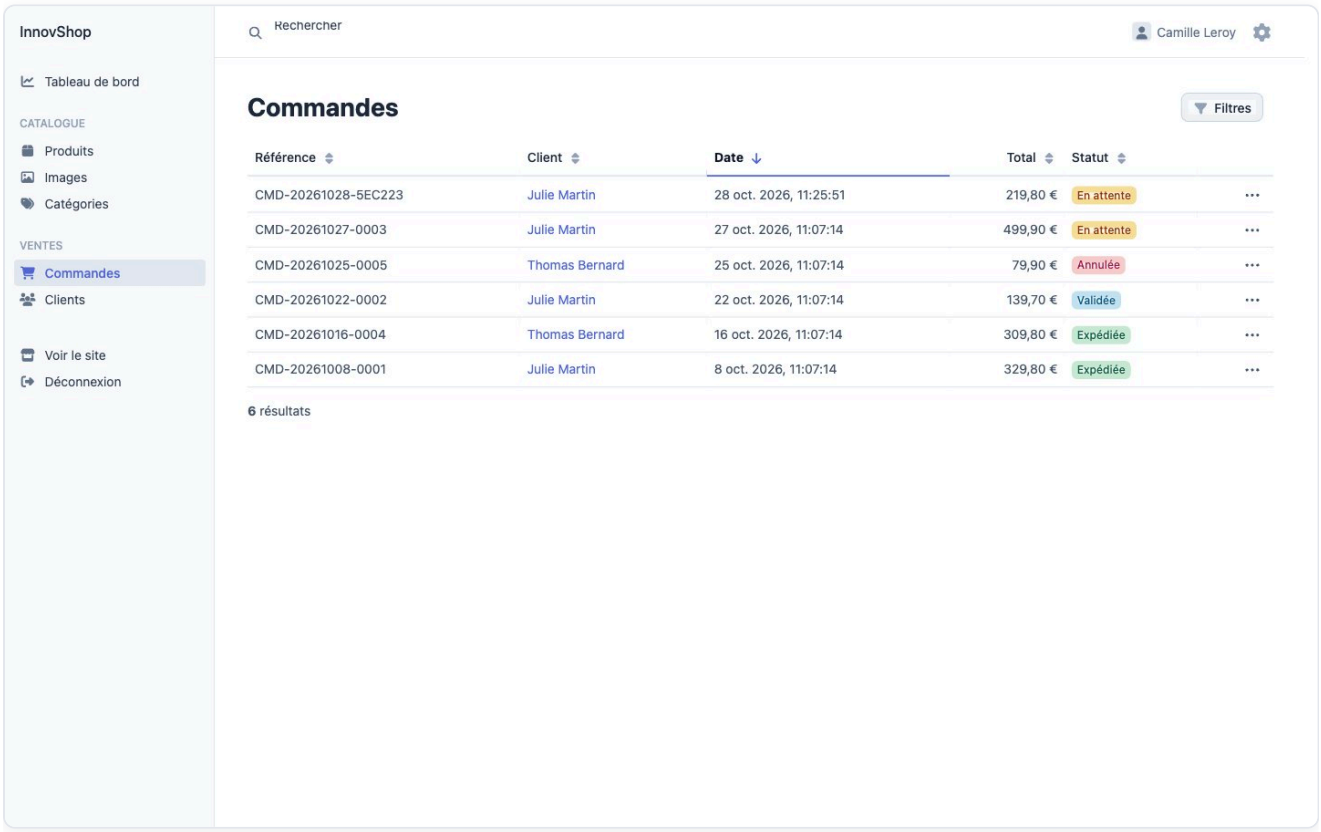
Capture 15 — Liste des produits dans EasyAdmin.



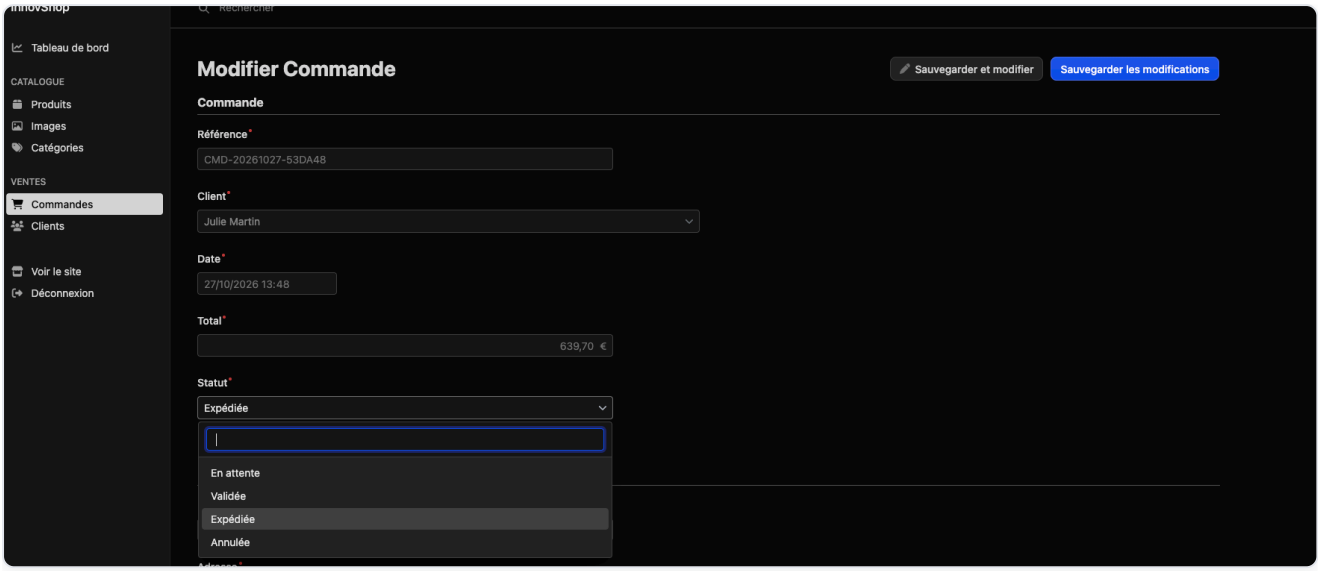
Capture 16 — Formulaire de la Montre Vita Sport avec ses deux options (taille du boîtier 42 mm et 46 mm).



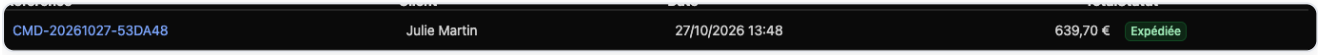
Capture 17 — La contrainte MaxFeatured refuse un 4e produit à la une : « Il y a déjà 3 produits à la une ».



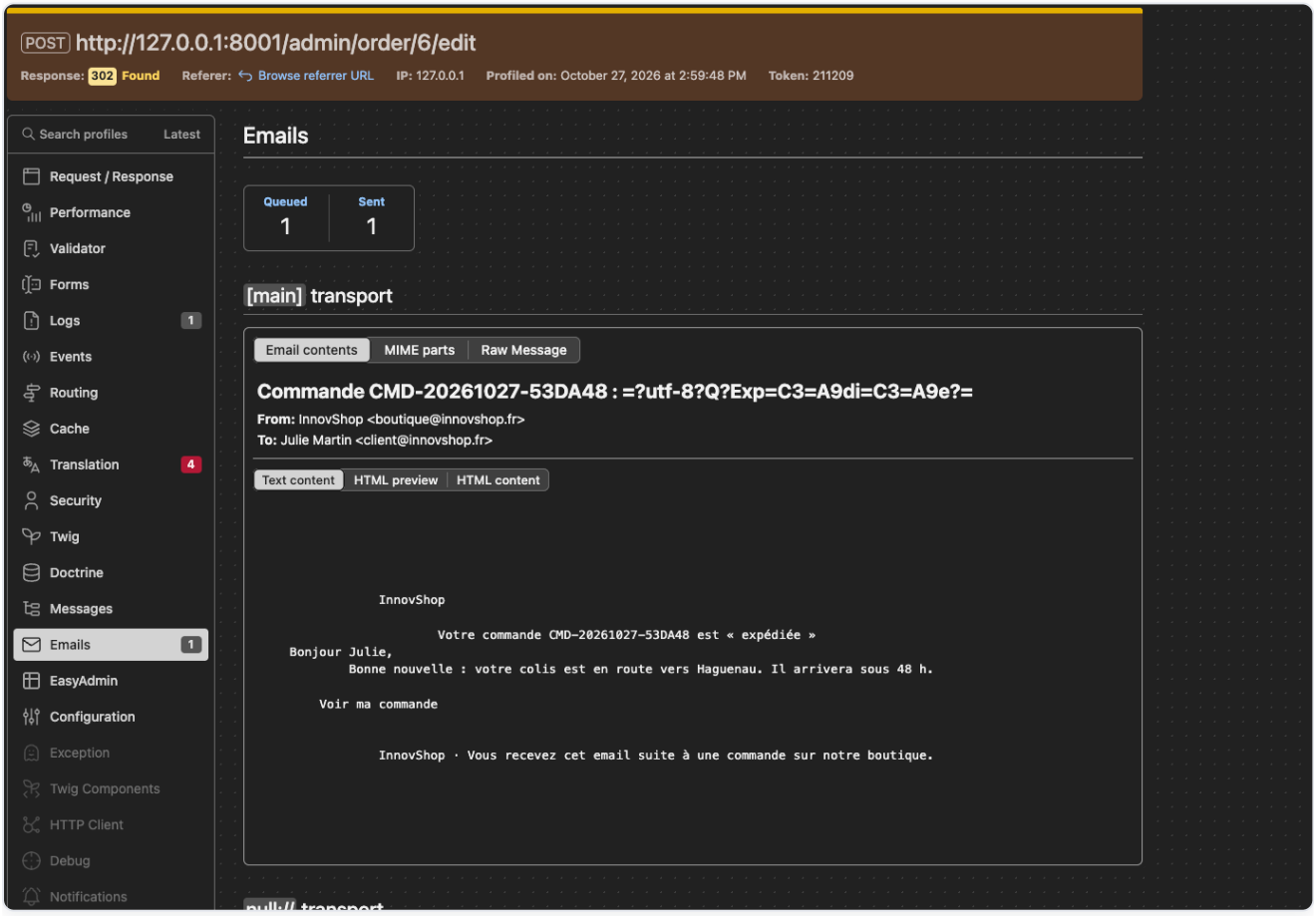
Capture 18 — Liste des commandes avec un badge de couleur par statut et le bouton Filtres.



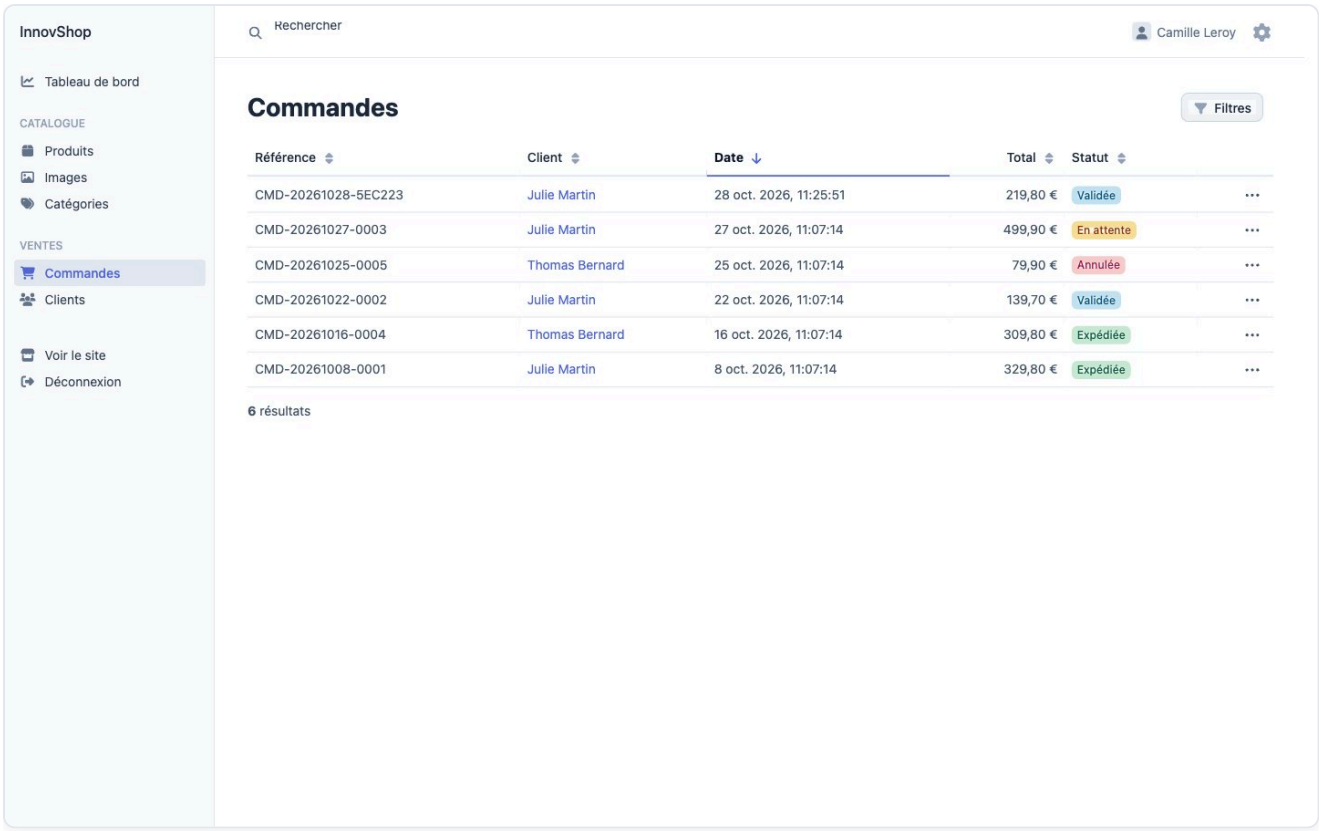
Capture 19 — Modification d'une commande : choix du nouveau statut.



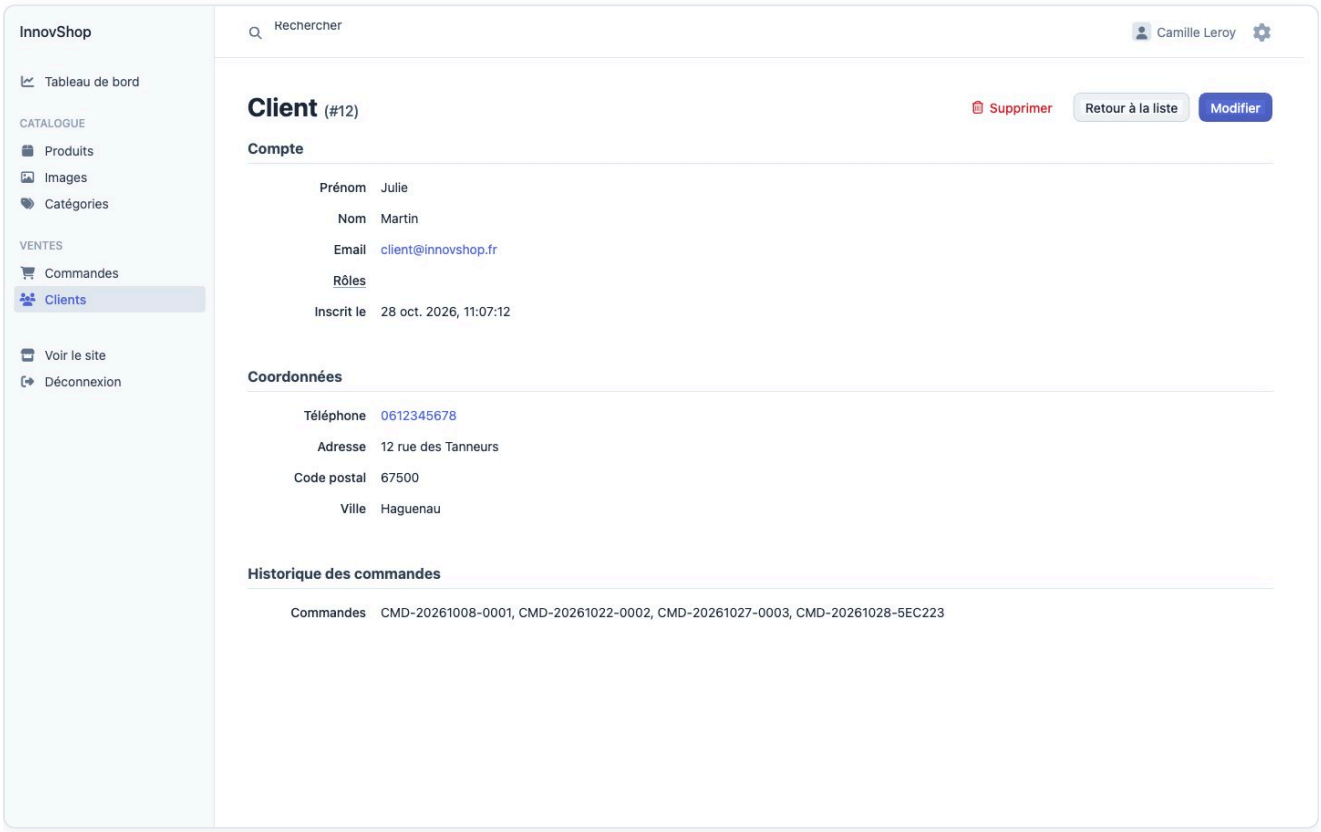
Capture 19 bis — Après enregistrement, la commande apparaît « Expédiée » dans la liste.



Capture 19 ter — Email envoyé automatiquement au client par le listener Doctrine, vu dans le profiler.

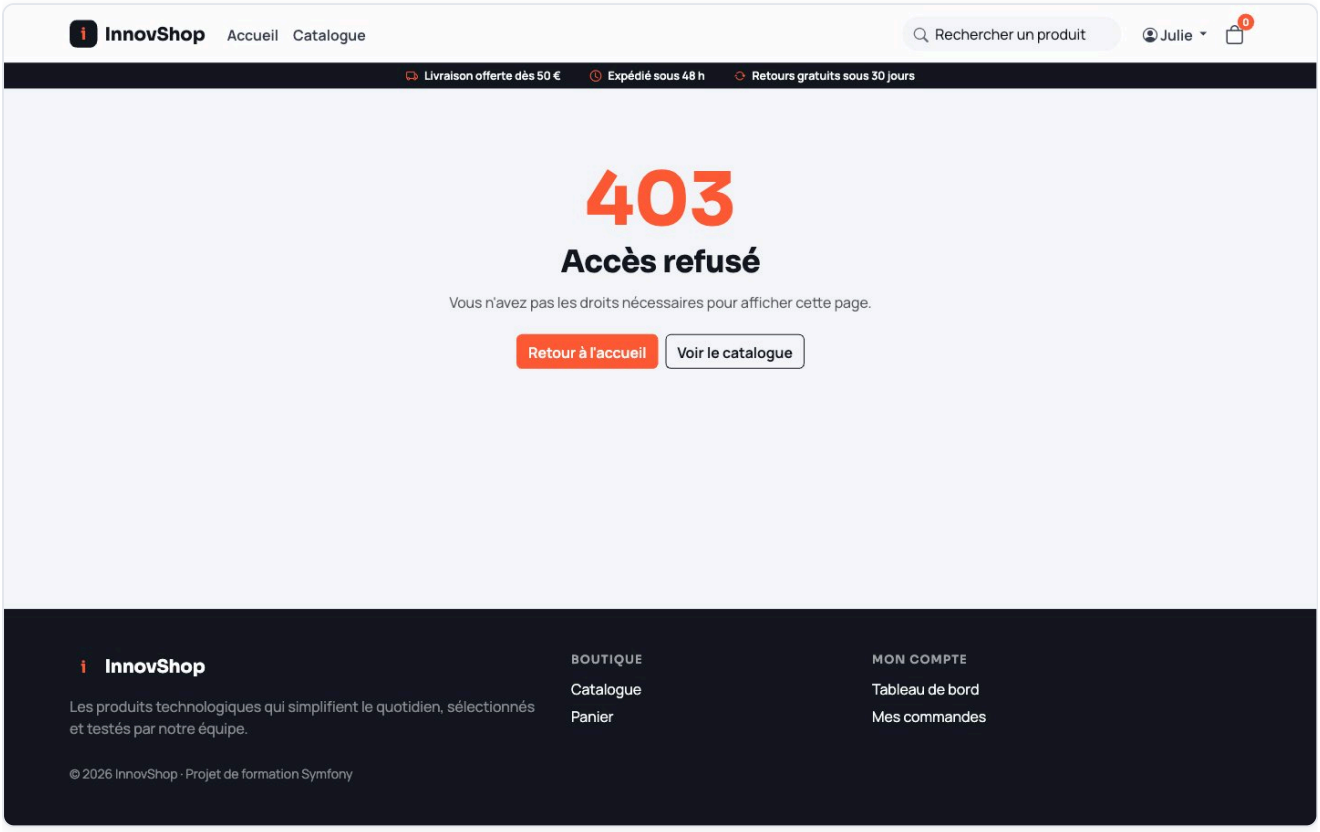


Capture 19 quater — Sur le site en ligne, la commande de test est passée de « En attente » à « Validée ».

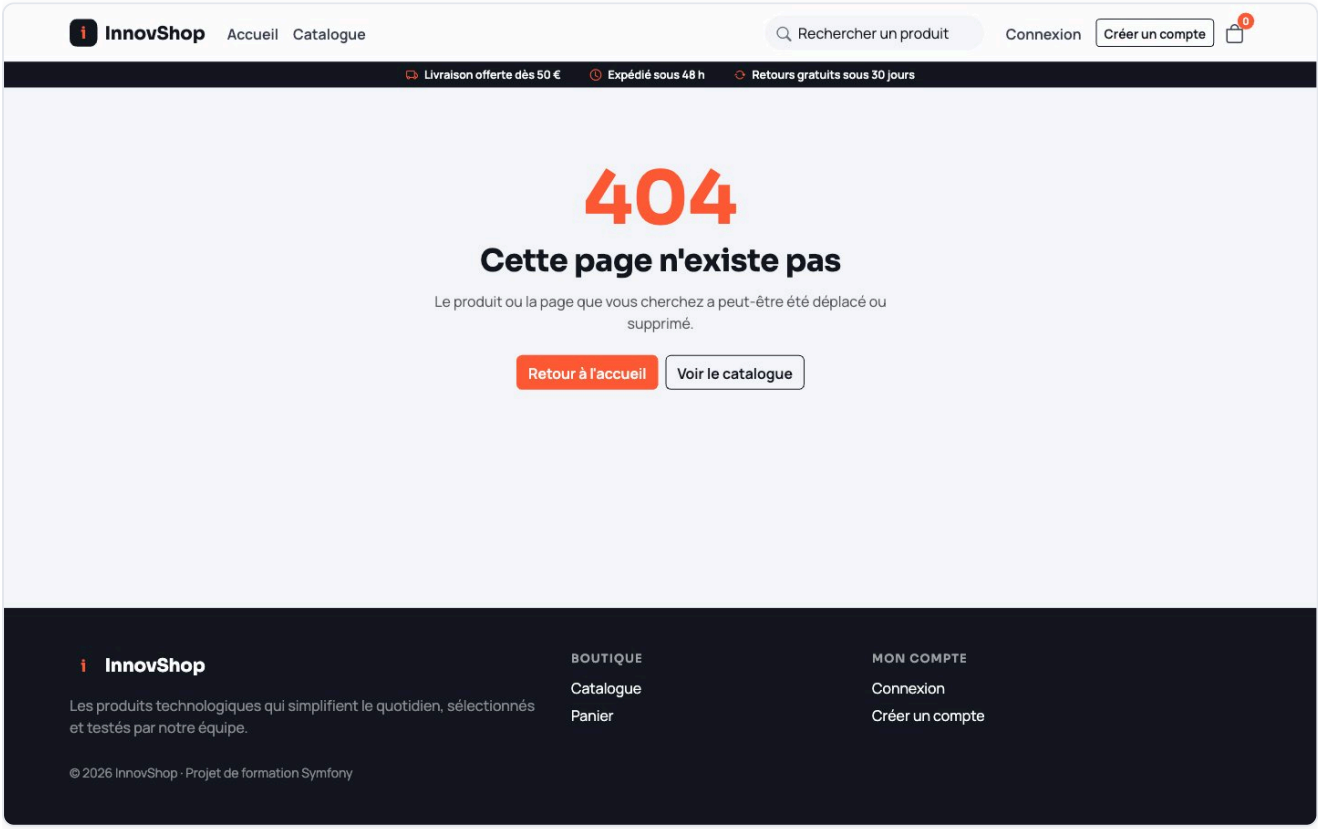


Capture 20 — Fiche client avec ses coordonnées et l'historique de ses commandes.

8.3 Sécurité



Capture 21 — Page 403 quand un client connecté tente d'ouvrir /admin.

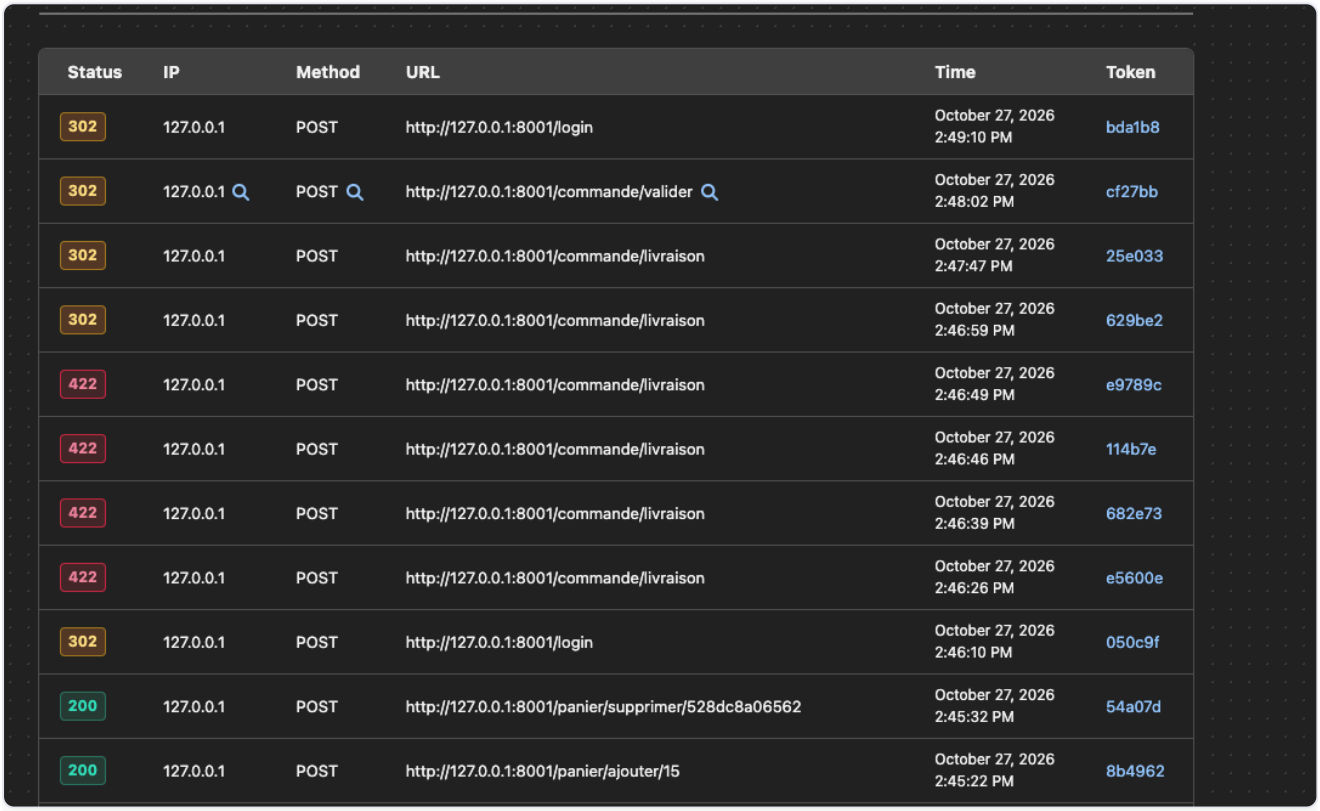


Capture 22 — Page 404 personnalisée.

9. Tests

9.1 Tests manuels

Avant chaque commit, j'ai testé le parcours complet dans le navigateur, en visiteur, en client et en administrateur : navigation, filtres, ajout et suppression au panier, commande avec des données valides puis invalides, consultation de l'espace client, changement de statut dans le back-office. Le **profiler Symfony** m'a servi à contrôler les requêtes SQL, les codes de réponse HTTP (422 quand un formulaire est invalide, 302 après une validation réussie) et les emails envoyés.



Status	IP	Method	URL	Time	Token
302	127.0.0.1	POST	http://127.0.0.1:8001/login	October 27, 2026 2:49:10 PM	bda1b8
302	127.0.0.1	POST	http://127.0.0.1:8001/commande/valider	October 27, 2026 2:48:02 PM	cf27bb
302	127.0.0.1	POST	http://127.0.0.1:8001/commande/livraison	October 27, 2026 2:47:47 PM	25e033
302	127.0.0.1	POST	http://127.0.0.1:8001/commande/livraison	October 27, 2026 2:46:59 PM	629be2
422	127.0.0.1	POST	http://127.0.0.1:8001/commande/livraison	October 27, 2026 2:46:49 PM	e9789c
422	127.0.0.1	POST	http://127.0.0.1:8001/commande/livraison	October 27, 2026 2:46:46 PM	114b7e
422	127.0.0.1	POST	http://127.0.0.1:8001/commande/livraison	October 27, 2026 2:46:39 PM	682e73
422	127.0.0.1	POST	http://127.0.0.1:8001/commande/livraison	October 27, 2026 2:46:26 PM	e5600e
302	127.0.0.1	POST	http://127.0.0.1:8001/login	October 27, 2026 2:46:10 PM	050c9f
200	127.0.0.1	POST	http://127.0.0.1:8001/panier/supprimer/528dc8a06562	October 27, 2026 2:45:32 PM	54a07d
200	127.0.0.1	POST	http://127.0.0.1:8001/panier/ajouter/15	October 27, 2026 2:45:22 PM	8b4962

Figure — Historique du profiler pendant un test du tunnel : les réponses 422 sont les envois du formulaire de livraison refusés par la validation, puis le 302 confirme l'envoi accepté et la validation de la commande.



Status	IP	Method	URL	Time	Token
302	127.0.0.1	POST	http://127.0.0.1:8001/admin/order/6/edit	October 27, 2026 2:59:48 PM	211209

Figure — Requête POST du changement de statut dans l'admin. Son profil contient l'email envoyé (capture 19 ter).

9.2 Tests automatisés avec PHPUnit

Type	Fichier	Ce qui est vérifié
Unitaire	`PriceFilterTest`	Conversion des centimes en euros (`1999` → « 19,99 € », séparateur des milliers)
Unitaire	`OrderStatusTest`	Chaque statut a un libellé français et une couleur ; correspondance avec la valeur stockée
Unitaire	`ProductImageTest`	Choix de l'image principale et de l'image liée à une option
Fonctionnel	`PublicPagesTest`	Accueil (3 + 3 produits), recherche du catalogue, fiche produit, erreur 404
Fonctionnel	`AccessControlTest`	Redirection des visiteurs vers la connexion, 403 pour un client sur /admin, accès admin, voter sur les commandes, ajout au panier en JSON

Les tests fonctionnels utilisent une base dédiée (`innovshop_test`), remplie avec les fixtures, pour ne jamais toucher aux données de développement. Exemple :

```
public function testCustomerCannotAccessBackOffice(): void
{
    $client = static::createClient();
    $client->loginUser($userRepository->findOneBy(['email' => 'client@innovshop.fr']));

    $client->request('GET', '/admin');
    $this->assertResponseStatusCodeSame(403);
}
```

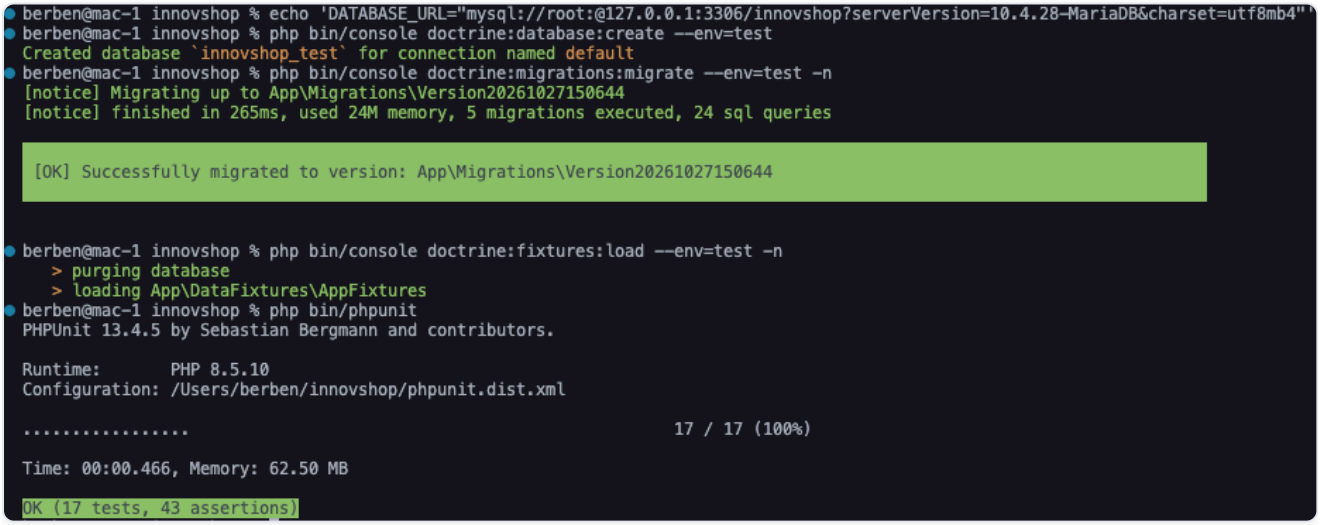


Figure — Préparation de la base de test (création, migrations, fixtures) puis lancement de `php bin/phpunit` : 17 tests, 43 assertions, tous au vert.

10. Déploiement

10.1 Hébergement

L'application est hébergée chez **Alwaysdata**, hébergeur français dont l'offre gratuite propose PHP, une base MariaDB, un accès SSH et un certificat HTTPS.

- Adresse du site : https://berben.alwaysdata.net/innovshop/
- Base de données : `berben_innovshop` sur `mysql-berben.alwaysdata.net`

10.2 Préparation du projet

- Fichier `public/.htaccess` : Alwaysdata utilise Apache ; ce fichier renvoie toutes les URL vers `index.php` pour que le routeur de Symfony les traite.
- Envoi des emails en direct, sans file d'attente Messenger, pour ne pas avoir de processus à maintenir sur le serveur.
- Données de démonstration disponibles aussi en production (photos des produits versionnées dans `fixtures/images/`, allégées à environ 1,2 Mo au total).

10.3 Étapes de mise en production

1. Création de la base de données et activation de l'accès SSH dans l'interface Alwaysdata, choix de PHP 8.4 ou plus.
2. Connexion en SSH, puis récupération du code : `git clone https://github.com/Mberben/innovshop.git`.
3. Création du fichier `.env.local` de production (jamais versionné) :

```
APP_ENV=prod
APP_SECRET=<chaîne aléatoire>
DATABASE_URL="mysql://berben:<mot de passe>@mysql-berben.alwaysdata.net:3306/berben_innovshop?
serverVersion=<version>-MariaDB&charset=utf8mb4"
MAILER_DSN=<serveur SMTP>
```

4. Installation et initialisation :

```
composer install --no-dev --optimize-autoloader
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console doctrine:fixtures:load --no-interaction
php bin/console importmap:install
php bin/console asset-map:compile
php bin/console cache:clear
```

5. Configuration du site dans Alwaysdata : type PHP, racine du site sur le dossier `public/`.
6. Vérification en ligne : inscription, commande, espace client, back-office, emails, pages d'erreur.

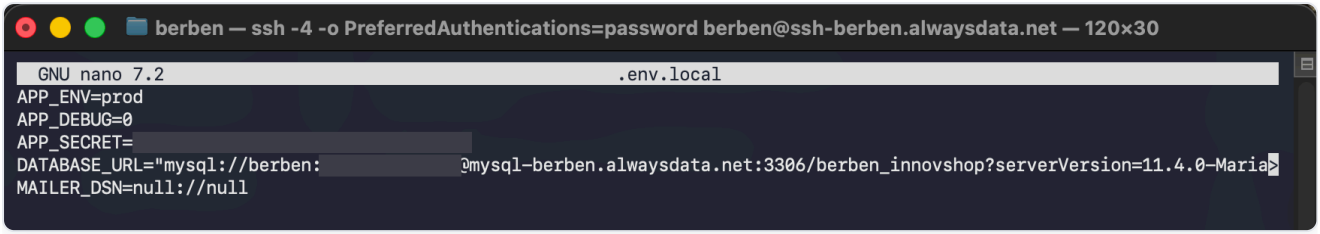


Figure — Session SSH sur Alwaysdata : création du fichier `.env.local` de production avec nano (APP_ENV=prod, APP_DEBUG=0). La clé secrète et le mot de passe de la base sont masqués sur la capture.

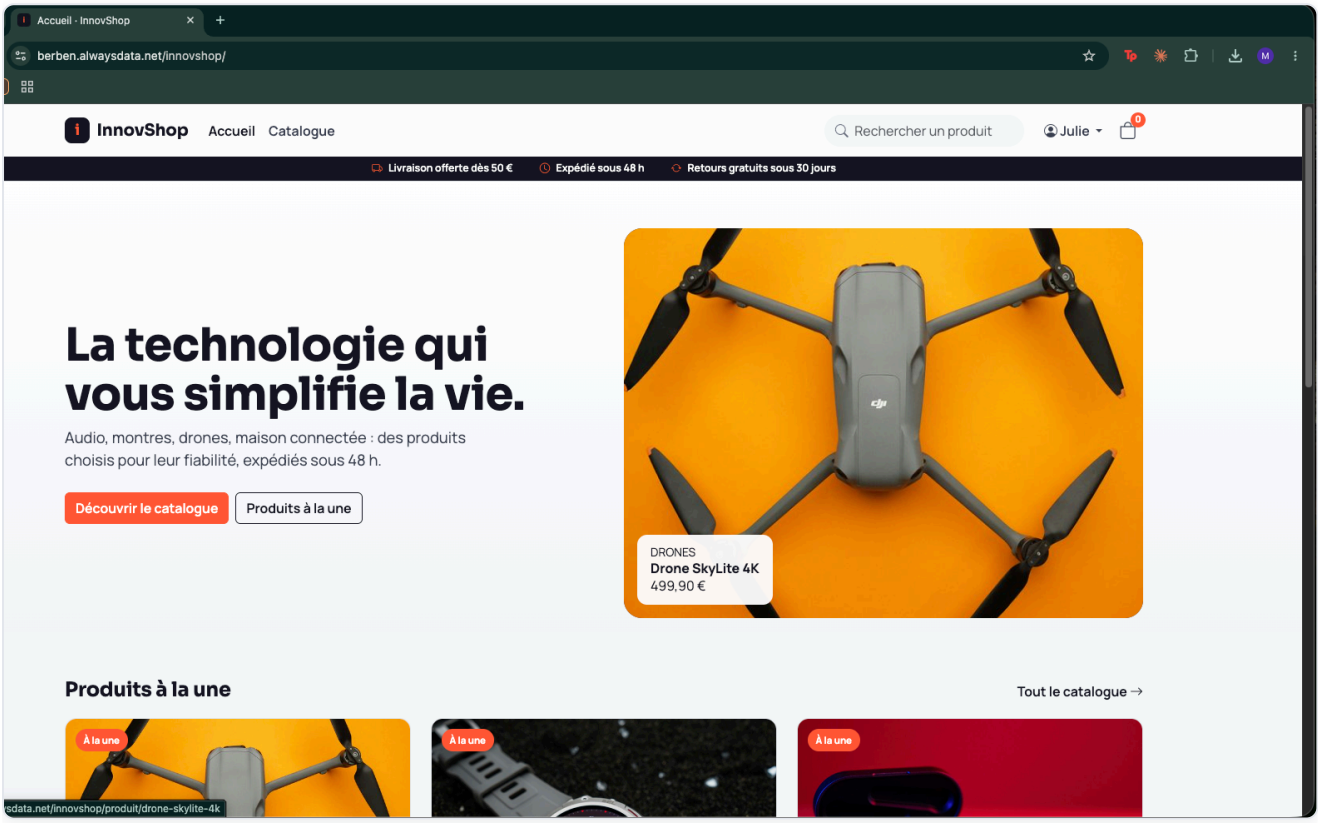


Figure — InnovShop en ligne à l'adresse berben.alwaysdata.net/innovshop, client connecté.

11. Bilan

11.1 Difficultés rencontrées et solutions

Difficulté	Cause	Solution
Symfony se connectait à PostgreSQL au lieu de MySQL	Le fichier `.env.local` était vide, donc la valeur par défaut de `.env` s'appliquait	Remplir `.env.local` ; vérifier avec `debug:dotenv`
Erreur « Case mismatch » au chargement de l'énumération	Dossier nommé `enum` alors que le namespace est `App\Enum`	Renommer le dossier en respectant la casse (autoloading PSR-4)
Propriété `order` refusée par Maker	ORDER est un mot réservé SQL	Renommer la propriété en `customerOrder`
Erreur Doctrine « A new entity was found through the relationship »	Les options créées dans le formulaire produit n'étaient pas enregistrées avec lui	Ajouter `cascade: ['persist']` sur les collections
Upload d'image impossible dans le formulaire produit	L'upload de fichiers dans un sous-formulaire EasyAdmin est peu fiable	Créer un écran « Images » dédié
Possibilité de mettre plus de 3 produits à la une	Aucune règle ne limitait la case « à la une »	Contrainte de validation personnalisée `MaxFeatured` ; interrupteur retiré de la liste
Photos très lourdes (38 Mo au total)	Photos téléchargées en haute définition	Redimensionnement à 1200 px et compression (1,2 Mo au total)
Prix et calculs	Risque d'erreurs d'arrondi avec des nombres décimaux	Stockage en centimes (entiers)

11.2 Compétences développées

- **Conception** : analyse d'un cahier des charges, rédaction de user stories, priorisation, modélisation des données.
- **Back-end** : Symfony, Doctrine (entités, relations, migrations, QueryBuilder), formulaires et validation, sécurité (authentification, rôles, voter), services, événements Doctrine, emails, génération de PDF.
- **Front-end** : Twig, Bootstrap, design responsive, JavaScript (fetch, manipulation du DOM).
- **Méthode et outils** : Kanban, Git et GitHub, tests PHPUnit, débogage avec le profiler, déploiement.

11.3 Évolutions possibles

- Paiement en ligne sécurisé (Stripe) et gestion des stocks.
- Quantités dans le panier et sauvegarde du panier en base pour le retrouver sur un autre appareil.

- Vérification de la cohérence code postal / ville avec l'API officielle des communes (geo.api.gouv.fr).
- Avis clients, codes promotionnels, liste d'envies.
- Recherche instantanée pendant la saisie et pagination du catalogue.
- Intégration continue : exécution automatique des tests à chaque push (GitHub Actions).

12. Conclusion

InnovShop répond à l'ensemble du cahier des charges : catalogue avec options, panier en temps réel, commande en trois étapes avec email, espace client et back-office complet. Plusieurs fonctionnalités vont au-delà : recherche et filtres, facture PDF, notification au changement de statut, tableau de bord avec statistiques, images liées aux options et tests automatisés.

Ce projet m'a permis de construire une application complète, de la conception au déploiement, avec un framework professionnel. J'ai surtout appris à structurer le code pour qu'il reste lisible et sûr : logique métier dans des services, validation au plus près des données, contrôle des accès, et données de commande figées pour garantir un historique fiable.

Les difficultés rencontrées (configuration de l'environnement, relations Doctrine, limites d'EasyAdmin) m'ont appris à lire les messages d'erreur, à utiliser les outils de débogage et à chercher la cause d'un problème plutôt que de contourner ses symptômes. La prochaine étape naturelle serait d'ajouter le paiement en ligne pour faire d'InnovShop une boutique prête à vendre.